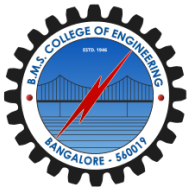
B.M.S. COLLEGE OF ENGINEERING

(Autonomous College under VTU)

Bull Temple Road, Basavangudi, Bangalore - 560019



Bio Inspired Algorithms AAT on

Course Code: **23DS7BSBIO**

Plant Disease Detection using Transfer-Learning CNN Optimized with Differential Evolution

in fulfillment of the requirements for the award of degree

BACHELOR OF ENGINEERING

in

Computer Science & Engineering (Data Science)

by

Aryanraj S Singh	**1BM23CD008**
Richik Chaudhuri	**1BM23CD050**
Suraj B	**1BM23CD061**
Sushant Deo	**1BM23CD062**

Under the Guidance of

Manoj Kumar S

Assistant Professor

Department of CSE (Data Science), B.M.S. College of Engineering

AY 2026–2027

Team 10

B.M.S. COLLEGE OF ENGINEERING

(Autonomous College under VTU)

Department of Computer Science & Engineering (Data Science)

CERTIFICATE

This is to certify that the project entitled "Plant Disease Detection using Transfer-Learning CNN Optimized with Differential Evolution" is a bona fide work carried out by the following students for the course Bio Inspired Algorithms (Course Code: **23DS7BSBIO**). It is certified that all corrections and suggestions indicated during the internal assessments have been incorporated in the final report submitted to the departmental library. The project report has been approved as it meets the academic requirements prescribed for the Bachelor of Engineering Degree in respect of project work.

Sl. No.	Student Name	USN	Student's Signature
1	Aryanraj S Singh	1BM23CD008	
2	Richik Chaudhuri	1BM23CD050	
3	Suraj B	1BM23CD061	
4	Sushant Deo	1BM23CD062	

Signature of the Guide

Manoj Kumar S
Assistant Professor

Signature of the HOD

Dr. Shyamala G
Professor & HOD

Examiners

	Name of Examiner	Signature
1		
2		

Academic Year 2026–2027

Acknowledgement

We thank the Principal of B.M.S. College of Engineering and the management of B.M.S. Educational Trust (BMSET) for providing the academic environment and facilities for this work.

We express our gratitude to **Dr. Shyamala G, Professor & Head of the Department, Computer Science and Engineering (Data Science)**, for departmental support. We thank our guide **Manoj Kumar S, Assistant Professor**, for guidance in the Bio-Inspired Algorithms course and for encouraging clear connections between algorithm design, implementation, and experimental evidence.

We also thank the department faculty, the maintainers of the open-source scientific software used in this project, and the researchers who made PlantVillage and the cited research openly accessible. Their work made this project possible.

Aryanraj S Singh

Richik Chaudhuri

Suraj B

Sushant Deo

Team 10

Abstract

Tomato plants suffer from many leaf diseases, and spotting them early helps farmers protect their crop. This project builds a computer model that looks at a photo of a tomato leaf and names one of ten classes (nine diseases or healthy). We use EfficientNet-B0, a compact image-recognition network already trained on millions of everyday photos (ImageNet), and adapt it to tomato leaves. Reusing a trained network in this way is called transfer learning.

How well such a model learns depends on its *hyperparameters*: the training settings we choose before training, such as the learning rate and how much of the network is retrained. Instead of guessing them, we search for them with *Differential Evolution (DE)*, a search method inspired by natural evolution that improves a group of candidate settings over generations. We wrote DE ourselves in NumPy and first checked it on five standard test functions against four other search methods.

The cleaned PlantVillage tomato dataset has 18,146 images, split into 12,701 for training, 2,723 for validation (used to choose settings), and 2,722 for the final test. We compare three models: a fixed default recipe (the baseline), the DE-tuned recipe, and a recipe found by random search with the same number of trials.

DE raises average test accuracy from 97.43% to 99.85% and macro-F1 (the average score across all disease classes) from 96.60% to 99.80%. This removes 94.29% of the baseline's mistakes, and every class improves. A statistical test (McNemar, $p = 9.048 \times 10^{-15}$) confirms that the improvement is not due to chance. Random search also reaches 99.85%, which confirms that the gain comes from tuning the training recipe. Heat-map explanations (Grad-CAM) and an offline web demo show what the models look at and how the search evolves.

Contents

Acknowledgement ii

Abstract iii

1 Introduction 1

2 Literature Review 2

3 Theoretical Background 4

4 Nature-Inspired Algorithms: Detailed Study 6

5 Methodology and Implementation 10

6 Experimental Setup and Test Functions 14

7 Results and Discussion 17

8 Comparative Performance Analysis 28

9 Applications and Case Studies 30

10 Conclusion 32

11 Future Work 33

12 References 34

13 GitHub Repo Link 37

List of Figures

4.1 A simple taxonomy of nature-inspired algorithms. BBO is biogeography-inspired; some methods fit more than one group. 6
4.2 DE mutation, bounce-back, and crossover shown on a real example from our code. 8

5.1 System pipeline 10
5.2 EfficientNet-B0 architecture and evolved decisions 11

6.1 Two-dimensional benchmark landscapes 16

7.1 Benchmark convergence across all runs 18
7.2 Distribution of final benchmark errors 18
7.3 DE population snapshots on Rastrigin 19
7.4 DE proxy-fitness progression 20
7.5 Population diversity and replacement success 20
7.6 Candidate fitness projected onto two genes 21
7.7 Parallel coordinates of candidate recipes 21
7.8 Gene–fitness associations 22
7.9 Final-training learning curves 23
7.10 Final held-out metric comparison 23
7.11 Paired seed-zero confusion matrices 24
7.12 Per-class F1 comparison 24
7.13 Classwise and macro ROC curves 25
7.14 Corruption stress test 26
7.15 Confidence reliability diagrams 26
7.16 Six held-out Grad-CAM cases 27

8.1 Equal-budget DE versus random search 28

List of Tables

2.1 Published results, each measured on its own dataset and protocol. 3

5.1 Number of images per class and split after cleaning. 11
5.2 Blocks, output shapes, and parameter counts of the baseline model. . . . 12
5.3 Baseline settings, fixed before the search. 12
5.4 The seven genes and how each is decoded into a training setting. Whole-number settings are rounded. 13

6.1 Computer used and DE search size. The starting population counts toward the total number of trials. 14
6.2 Training and evaluation settings for the neural-network experiments. . . . 14
6.3 Search range and location of the best point for each test function. 15

7.1 Final error on each test function: mean and standard deviation over all runs. Lower is better. 17
7.2 Baseline and the final recipes chosen by DE and random search. 19
7.3 Final test results: mean and standard deviation over three training seeds. 22
7.4 95% bootstrap confidence intervals for the seed-zero models. 22
7.5 Calibration error, average corruption accuracy, and speed of the seed-zero models. 25

8.1 Final benchmark errors: median and best over all runs. 29
8.2 Rank-sum tests comparing DE with each other method; negative z favours DE. 29
8.3 Model size and speed; training time and prediction time are mean and standard deviation across seeds. 29

13.1 Repository map. 37

CHAPTER 1

Introduction

1.1 Background and problem

Plant diseases and pests destroy a large share of the world's crops, and the losses differ by crop and region [1]. Tomato is one of India's major vegetable crops [2]. Many tomato diseases first show up as spots or colour changes on the leaves, so a photo of a leaf can help with early diagnosis. Studies on the PlantVillage image collection show that computer models can recognise such diseases accurately [3, 4].

Our model is a *convolutional neural network (CNN)*, a type of model that learns visual patterns such as edges, spots, and textures directly from images. We start from EfficientNet-B0, a CNN already trained on general photos, and adapt it to tomato leaves. The question this project answers is: *can Differential Evolution (DE) find better training settings for this network than a sensible default?*

To answer it, DE searches over seven settings: learning rate, two kinds of regularisation (ways to stop the model memorising the training images), the size of the final layer, how much of the network is retrained, and how strongly the training images are varied. Each candidate setting is scored by *macro-F1* on the validation images, which is the average score across all ten classes so that small classes count as much as large ones.

1.2 Objectives

1. Prepare a clean, well-documented dataset with fair training, validation, and test splits.
2. Implement DE from scratch, with tested mutation, crossover, and selection steps.
3. Compare a fixed baseline, DE, and random search with the same number of trials, choosing settings on validation data only.
4. Evaluate the final models over repeated training runs, per class, and with statistical tests and visual explanations.
5. Deliver an offline demo together with the report, slides, and reproducible code.

1.3 Contribution and organisation

The main contribution is a complete, reproducible pipeline in which DE tunes a transfer-learning CNN and clearly improves on the baseline, together with a fair random-search comparison. The work combines established methods rather than proposing a new algorithm.

Chapters 2–4 cover earlier work, theory, and algorithms; Chapters 5–6 the implementation and setup; Chapters 7–8 the results; and the rest applications, conclusions, and future work.

CHAPTER 2

Literature Review

2.1 Image-based plant disease classification

Hughes and Salathé released the PlantVillage collection of labelled plant-leaf photos to support research on automatic diagnosis [3]. Mohanty et al. trained deep networks on it and reached very high accuracy, while noting that photos taken in real fields are harder [4]. Ferentinos tested several CNN designs on a larger multi-crop collection [5]. Too et al. compared fine-tuned networks and found DenseNet performed best [6]. Each of these studies uses its own classes and data splits, so their numbers are not directly comparable with ours.

Kamilaris and Prenafeta-Boldú review the wider use of deep learning in agriculture [7]. PlantDoc collects leaf photos taken outside the laboratory [8], and Gui et al. study how background and lighting in field photos affect recognition [9]. These works show that field photos are an important next step after laboratory datasets.

2.2 Transfer learning and efficient networks

EfficientNet grows a network in a balanced way by scaling its depth, width, and input image size together [10]. We use its smallest version, B0, and tune how it is trained. MobileNetV2 and MobileNetV3 are other compact networks designed for phones [11, 12].

Yosinski et al. show that early layers of a trained network learn general features, while later layers become specific to the original task [13]. This means the number of layers we retrain (the fine-tuning depth) is an important choice. Keeping the whole pretrained network fixed is a simple, common baseline. Retraining more layers can adapt better to leaves, but costs more time. We let the search decide.

2.3 Hyperparameter and evolutionary search

Grid search tries every combination from a fixed list of values; random search simply tries randomly chosen settings. Bayesian optimisation builds a model of which settings are likely to work well [14]. Hyperband saves time by stopping weak trials early [15]. Random search is a strong and simple competitor, so we use it as our control: it tells us whether DE's guided search adds value over picking the same number of settings at random.

Evolutionary methods have been used successfully to design and tune networks. Real et al. used an evolutionary method to design image classifiers [16]. Wang et al. used particle swarm optimisation and a hybrid DE method to design CNNs [17, 18]. DEHB combines DE with Hyperband for tuning [19]. These results motivate applying DE to our seven training settings.

Table 2.1: Published results, each measured on its own dataset and protocol.

Study	Dataset / model	Search or comparison	Accuracy
Mohanty et al. [4]	PlantVillage, 38 classes; AlexNet/GoogLeNet	Training/model variants	99.35%
Ferentinos [5]	58 categories; CNNs	CNN architectures	99.53%
Too et al. [6]	PlantVillage, 38 classes; DenseNet	Fine-tuned architectures	99.75%
Gui et al. [9]	Field-PV; adapted CNN	Background/features	72.03%

2.4 Research gap

Most earlier plant-disease studies choose training settings by hand. Few apply an evolutionary search to these settings and compare it fairly with a fixed baseline and with random search at the same cost. This project fills that gap for tomato leaves: it uses fair data splits, a baseline fixed in advance, the same number of trials for each search method, and repeated final training runs. It also links the algorithm to a working offline demo.

CHAPTER 3

Theoretical Background

3.1 CNNs and transfer learning

A CNN slides small learned filters over an image to detect local patterns such as edges and spots. Stacking many such layers lets the network recognise larger shapes and textures. EfficientNet-B0 is built from efficient blocks called MBConv, which include a small "attention" step (squeeze-and-excitation) that tells the network which feature channels matter most. EfficientNet scales depth d, width w, and image resolution r together as $d = \alpha^\phi$, $w = \beta^\phi$, $r = \gamma^\phi$ with $\alpha\beta^2\gamma^2 \approx 2$ [10]. We keep the B0 size.

On top of the network we place a small *classifier head*: dropout, an optional hidden layer, and ten outputs, one per class. The softmax function $p_c = \exp(z_c)/\sum_j \exp(z_j)$ turns the ten outputs z_c into probabilities that add up to one. Layers that are "frozen" keep their pretrained values; "unfrozen" layers are retrained on tomato leaves. Retraining later layers helps because they hold the most task-specific features [13].

3.2 Training and regularisation

Training minimises the cross-entropy loss $\mathcal{L} = -N^{-1}\sum_i \sum_c q_{ic} \log p_{ic}$, which is small when the model gives high probability to the correct class. Several settings help the model generalise instead of memorising:

- **Dropout** randomly switches off some connections during training.
- **Weight decay** keeps the network's weights small.
- **Label smoothing** softens the target answers slightly, $q_{ic} = (1-\epsilon)y_{ic} + \epsilon/C$, so the model is less over-confident.
- **Augmentation** shows randomly flipped, cropped, and colour-changed versions of each image.

The baseline uses the Adam optimiser; the searched recipes use AdamW, a version of Adam with built-in weight decay. The learning rate rises briefly at the start (warm-up) and then decreases smoothly (cosine schedule). Training stops early when validation macro-F1 stops improving, and the best checkpoint is kept.

3.3 Classification metrics

For class c, TP_c counts correct detections, FP_c false alarms, and FN_c missed cases:

$$P_c = \frac{TP_c}{TP_c + FP_c}, \qquad R_c = \frac{TP_c}{TP_c + FN_c}, \qquad F_{1,c} = \frac{2P_cR_c}{P_c + R_c}, \tag{3.1}$$

$$F_{1,\text{macro}} = \frac{1}{C}\sum_c F_{1,c}, \qquad \text{Accuracy} = \frac{1}{N}\sum_i \mathbf{1}[\hat{y}_i = y_i]. \tag{3.2}$$

Precision P_c asks "when the model says class c, how often is it right?"; recall R_c asks "how many real class-c leaves did it find?"; F1 balances the two. Macro-F1 averages F1 over all classes, so every disease counts equally. The ROC curve shows how well the model separates one class from the rest at every confidence threshold, and its area (AUC) is 100% for perfect separation.

3.4 Statistical checks

Bootstrap confidence intervals give a plausible range for a score by repeatedly re-sampling the test images [20]. The *McNemar test* compares two models on the same test images. It looks only at the images where exactly one model is right and checks whether this split could happen by chance [21]. A very small p-value means the difference is real.

3.5 Calibration, explanations, and robustness

A model is well *calibrated* when its confidence matches how often it is right. Expected calibration error (ECE) measures the gap, averaged over confidence bins B_m [22]:

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\right|. \tag{3.3}$$

Grad-CAM produces a heat map showing which parts of the image most influenced the prediction [23]. *Corruption tests* add blur, noise, or darkness to test images to see how the model copes with poorer photos [24].

CHAPTER 4

Nature-Inspired Algorithms: Detailed Study

4.1 What metaheuristics are

A *metaheuristic* is a general search strategy for problems where we cannot simply compute the best answer with a formula. Many are inspired by nature. *Population* methods keep a group of candidate solutions; *trajectory* methods improve a single one. Every method balances *exploration* (trying new regions) and *exploitation* (refining good regions already found).

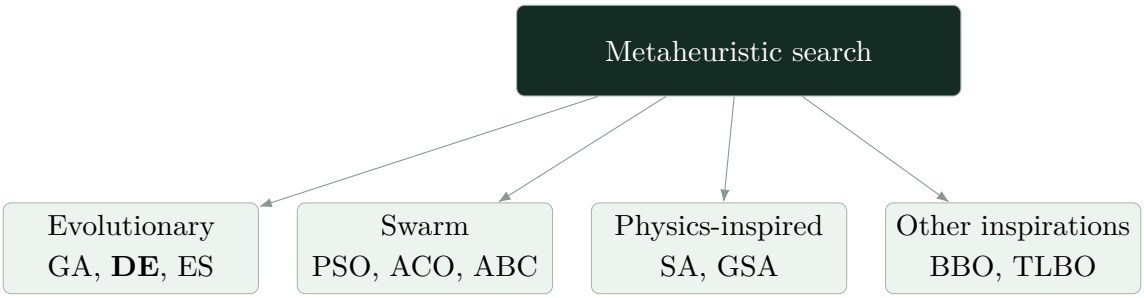


Figure 4.1: A simple taxonomy of nature-inspired algorithms. BBO is biogeography-inspired; some methods fit more than one group.

The genetic algorithm (GA) imitates natural selection and breeding [25]. Particle swarm optimisation (PSO) imitates a flock of birds that remembers good places [26]. Ant colony optimisation follows the trail-laying of ants [27]. Simulated annealing imitates slowly cooling metal [28].

4.2 How other algorithms update solutions

A real-coded GA mixes two parents, for example $\mathbf{u} = \alpha\mathbf{x}_a + (1-\alpha)\mathbf{x}_b$, then adds small random changes (mutation) and keeps the fittest. Our GA uses tournament selection, blend crossover, Gaussian mutation, and keeps the best solution each generation (elitism).

In PSO, each particle moves with a velocity pulled toward its own best position and the swarm's best position:

$$\mathbf{v}_i^{t+1} = w\mathbf{v}_i^t + c_1\mathbf{r}_1 \odot (\mathbf{p}_i - \mathbf{x}_i^t) + c_2\mathbf{r}_2 \odot (\mathbf{g} - \mathbf{x}_i^t), \tag{4.1}$$

$$\mathbf{x}_i^{t+1} = \mathbf{x}_i^t + \mathbf{v}_i^{t+1}, \tag{4.2}$$

where $\mathbf{p}_i$ is the particle's best position and $\mathbf{g}$ the swarm's best. In ant colony optimisation, an ant prefers a path in proportion to its pheromone τ_{ij}^{α} times its attractiveness η_{ij}^{β}; good paths receive more pheromone over time [27]. Simulated annealing sometimes accepts a worse solution, with probability $\exp(-\Delta E/T)$, so that it can escape local traps [28].

The firefly algorithm moves dimmer fireflies toward brighter ones: $\mathbf{x}_i \leftarrow \mathbf{x}_i + \beta_0 e^{-\gamma r_{ij}^2}(\mathbf{x}_j - \mathbf{x}_i) + \alpha\boldsymbol{\epsilon}$ [29]. BBO treats solutions as islands that exchange species, with immigration rate $\lambda(s) = I(1 - s/S)$ and emigration rate $\mu(s) = Es/S$ [30]. ABC imitates honey-bee foraging roles [31], and GWO imitates the hunting hierarchy of grey wolves [32]. GA, PSO, and the DE variant jDE are also used as comparison methods in our benchmark study.

4.3 Differential Evolution in depth

DE keeps a population of NP candidate vectors. In our project each vector holds seven numbers between 0 and 1, one for each training setting. Each generation, every member $\mathbf{x}_i$ (the *target*) tries to produce a better child in three steps [33, 34].

Step 1: Mutation. In the DE/rand/1 strategy,

$$\mathbf{v}_i = \mathbf{x}_{r_1} + F(\mathbf{x}_{r_2} - \mathbf{x}_{r_3}), \qquad r_1, r_2, r_3 \neq i. \tag{4.3}$$

In words: take one random member and push it along the difference between two other random members, scaled by F. Because the step comes from the population itself, steps are large when the population is spread out and small when it has converged.

If a mutated value falls outside $[0, 1]$, it is placed at a random point between the parent's value and the boundary it crossed:

$$v_{ij} \leftarrow x_{ij} + r(b - x_{ij}), \quad r \sim U(0, 1), \quad b \in \{0, 1\}. \tag{4.4}$$

In words: the value "bounces back" inside the allowed range instead of sticking to the edge.

Step 2: Binomial crossover.

$$u_{ij} = \begin{cases} v_{ij}, & r_j < CR \text{ or } j = j_{\text{rand}}, \\ x_{ij}, & \text{otherwise.} \end{cases} \tag{4.5}$$

In words: the child $\mathbf{u}_i$ takes each setting from the mutant with probability CR, otherwise from the parent; one randomly chosen setting j_{rand} always comes from the mutant, so the child is never an exact copy of the parent.

Step 3: Greedy selection.

$$\mathbf{x}_i^{g+1} = \begin{cases} \mathbf{u}_i, & f(\mathbf{u}_i) \geq f(\mathbf{x}_i^g), \\ \mathbf{x}_i^g, & \text{otherwise.} \end{cases} \tag{4.6}$$

In words: the child replaces its parent only if it scores at least as well, so the best score found never gets worse.

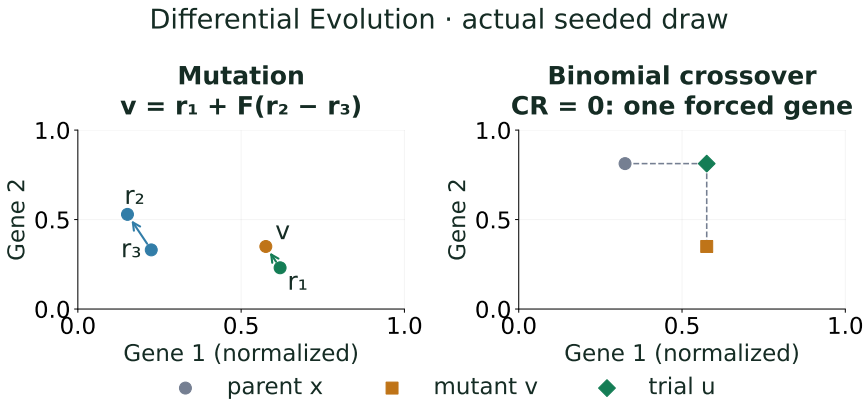


Figure 4.2: DE mutation, bounce-back, and crossover shown on a real example from our code.

4.4 Pseudocode and cost

Algorithm 1 DE/rand/1/bin as implemented in this project

Require: Fitness function f, dimension D, population size NP, generations G, F, CR

Spread NP starting vectors evenly over $[0, 1]^D$ (Latin-hypercube sampling)
Score every starting vector and save progress
for $g = 1, \ldots, G$ **do**
    $S \leftarrow X$ ▷ Copy of this generation, used to pick donors
    for $i = 1, \ldots, NP$ **do**
        Pick three different members r_1, r_2, r_3, none equal to i
        $v \leftarrow S_{r_1} + F(S_{r_2} - S_{r_3})$ ▷ Mutation
        Bounce any out-of-range value back inside $[0, 1]$
        Build child u by binomial crossover with forced j_{rand}
        Score $f(u)$
        if $f(u) \geq f(X_i)$ **then**
            $X_i \leftarrow u$ ▷ Greedy selection
        end if
        Save progress so the search can resume after an interruption
    end for
    Record best and mean fitness, diversity, and success rate
end for
return best vector found and the full history

The total number of trials is $NP(G + 1)$, counting the starting population. The DE arithmetic itself is cheap ($O(GNPD)$); almost all the time goes into training a network for each trial. We track *diversity* (how spread out the population is) and *success rate* (how many children beat their parents) to see whether the search is still making progress.

4.5 Variants and control settings

Other DE strategies change the mutation: DE/best/1 starts from the current best member, DE/rand/2 uses two difference vectors, and current-to-best moves each member toward the best [35]. A larger F gives bigger steps (more exploration); a larger CR mixes in more mutant settings. jDE lets each member adapt its own F and CR [36]. SHADE learns good settings from past successes, and L-SHADE also shrinks the population over time [37, 38]. We use classic DE for the CNN and include jDE in the benchmark study.

Why DE for this problem. Our seven settings are all mapped to numbers between 0 and 1, which suits DE well. DE needs no gradients, has only three control settings (NP, F, CR), and its trials can run independently. Its value is checked fairly against random search with the same number of trials [39].

CHAPTER 5

Methodology and Implementation

5.1 Overall pipeline

The work runs in four stages: prepare the data, search for the best settings, retrain the chosen recipes, and test them once on held-out images. Each stage is a numbered Python script, and a few small shared modules hold the data, model, training, and optimiser code. The offline demo uses the saved results and trained models.

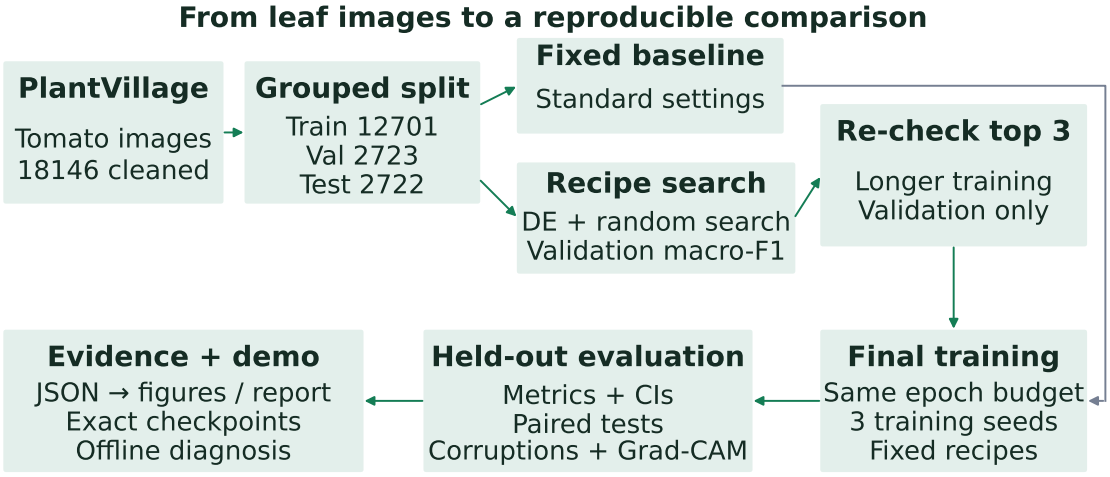


Figure 5.1: The project pipeline, from data preparation to search, final evaluation, and the offline demo.

5.2 Data preparation and splitting

We downloaded the colour tomato images from the public PlantVillage repository. The ten class folders, sorted alphabetically, define the class labels. Every image was checked to open correctly and converted to standard RGB colour. Exact duplicate images were removed. Images that look almost identical (for example, two photos of the same leaf) were grouped together and always placed in the same split, so the model is never tested on a near-copy of a training image.

The raw data had 18,160 images. We removed 14 duplicates and found 0 broken files, leaving 18,146 images. These were split about 70/15/15 into training, validation, and test sets, keeping the class proportions the same in each split. The split lists are saved in the repository so every run uses exactly the same images.

Table 5.1: Number of images per class and split after cleaning.

Class	Original	Train	Validation	Test
Bacterial spot	2,127	1,489	319	319
Early blight	1,000	700	150	150
Late blight	1,909	1,331	285	285
Leaf Mold	952	666	143	143
Septoria leaf spot	1,771	1,239	266	266
Spider mites	1,676	1,174	251	251
Target Spot	1,404	982	211	211
Yellow Leaf Curl Virus	5,357	3,750	804	803
mosaic virus	373	261	56	56
healthy	1,591	1,109	238	238
Total	18,160	12,701	2,723	2,722

5.3 Image preparation

Images are resized once and cached to make training faster. During training, each image is randomly cropped and flipped; the searched recipes can also add RandAugment, which applies random colour and shape changes of a chosen strength. The baseline uses only cropping and flipping. Validation and test images are simply resized and centre-cropped, with no randomness.

5.4 Model and baseline

We use EfficientNet-B0 with ImageNet weights and the classifier head from Chapter 3. The network body has nine blocks. The *fine-tuning depth* k says how many of the last blocks are retrained: $k = 0$ trains only the head, and $k = 9$ retrains the whole network. The baseline trains 330,506 parameters.

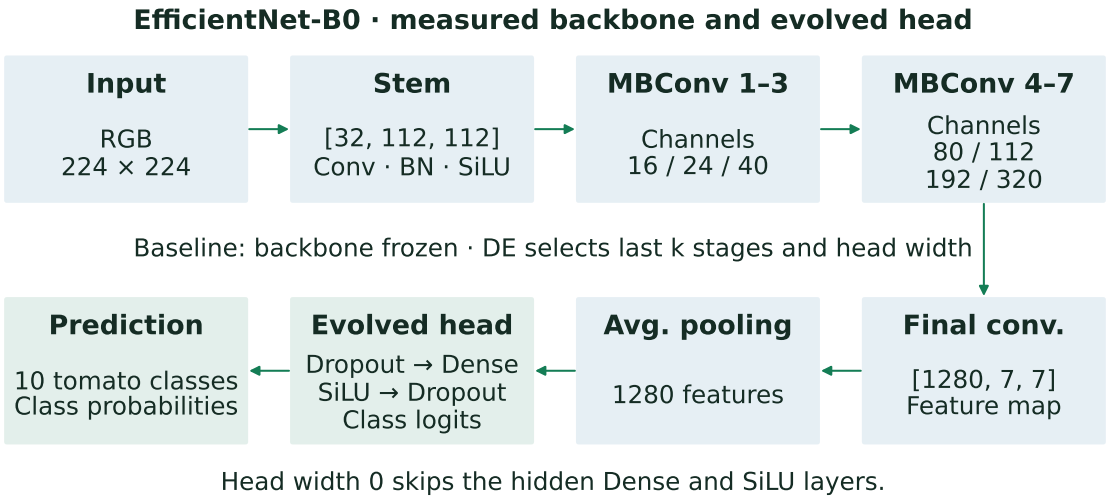


Figure 5.2: EfficientNet-B0 with our classifier head, showing which parts the DE chromosome controls.

Table 5.2: Blocks, output shapes, and parameter counts of the baseline model.

Block	Output shape	Parameters	Trainable (baseline)
0	$1 \times 32 \times 112 \times 112$	928	0
1	$1 \times 16 \times 112 \times 112$	1,448	0
2	$1 \times 24 \times 56 \times 56$	16,714	0
3	$1 \times 40 \times 28 \times 28$	46,640	0
4	$1 \times 80 \times 14 \times 14$	242,930	0
5	$1 \times 112 \times 14 \times 14$	543,148	0
6	$1 \times 192 \times 7 \times 7$	2,026,348	0
7	$1 \times 320 \times 7 \times 7$	717,232	0
8	$1 \times 1280 \times 7 \times 7$	412,160	0
classifier	1×10	330,506	330,506

The baseline is a common "default" transfer-learning recipe. It was fixed before any search was run and was never changed afterwards.

Table 5.3: Baseline settings, fixed before the search.

Setting	Value
Backbone	ImageNet-pretrained EfficientNet-B0
Unfrozen feature blocks	0
Classifier hidden units	256
Dropout	0.2
Optimizer / learning rate	Adam / 10^{-3}
Weight decay / label smoothing	0 / 0
Extra RandAugment	Disabled
Batch / final resolution	32 / 224×224
Epoch ceiling / patience	15 / 4

5.5 Chromosome and fitness

Each DE member is a *chromosome* of seven numbers between 0 and 1. A decoding rule turns each number into a real setting:

The *fitness* of a chromosome is the validation macro-F1 of a network trained with its settings. To keep the search affordable, each trial uses a quick training run: smaller images, part of the training set, and a few epochs. All trials use the same data and random seed, so they are compared fairly. If two chromosomes decode to the same settings, the saved score is reused. A trial that crashes receives a fitness of zero and the search continues.

The DE population starts from evenly spread random points; the baseline is not inserted into it. After the search, the best few recipes are retrained at full image size, and the one with the highest validation macro-F1 is chosen. Random search follows exactly the same rules and number of trials.

Table 5.4: The seven genes and how each is decoded into a training setting. Whole-number settings are rounded.

Gene	Decision	Decoded range
x_1	Learning rate	$10^{-4.5+2.5x_1}$
x_2	Weight decay	10^{-6+4x_2}
x_3	Dropout	$0.6x_3$
x_4	Hidden units	Index into $\{0, 128, 256, 512\}$
x_5	Unfrozen blocks	Integer in $\{0, \ldots, 9\}$
x_6	Label smoothing	$0.2x_6$
x_7	RandAugment magnitude	Integer in $\{0, \ldots, 15\}$

5.6 Saving progress and the demo

The search saves its progress after every trial, so it can resume after an interruption without losing work. Each trained model is saved with its settings, seed, and best epoch.

The demo loads the baseline and DE models and makes real predictions on uploaded or example leaves, with Grad-CAM heat maps. It checks uploaded files and rejects unsupported ones. After installation, it runs fully offline.

CHAPTER 6

Experimental Setup and Test Functions

6.1 Hardware and software

Table 6.1 lists the computer used and the size of the DE search. A short speed test at the start of the project was used to choose a search size (tier B) that fits the available time.

Table 6.1: Computer used and DE search size. The starting population counts toward the total number of trials.

Item	Measured / configured value
CPU	Apple M4 Pro
Memory (GiB)	48
Device	mps
Python	3.11.14
PyTorch	2.15.0
Operating system	macOS-26.7.1-arm64-arm-64bit
Compute tier	B

Setting	Value
Proxy resolution	128
Training fraction	20%
Proxy epochs	3
DE population	8
DE generations	8
Budget per search	72
Scale F	0.6
Crossover CR	0.9
Search seed	42

6.2 Neural-network protocol

Table 6.2: Training and evaluation settings for the neural-network experiments.

Stage	Configured / frozen protocol
Data partition	Train / validation / test: 12,701 / 2,723 / 2,722; grouped split seed 42.
Proxy search	128 px; 20% of training data; 3 epochs; 72 candidate evaluations per method.
Search seeds	Proposal generation: DE 42, random search 43; shared fitness-training seed 42.
Validation reranking	Top 3 distinct candidates per method; 224 px; full training split; 3 epochs; training seed 42.
Final training	224 px; full training split; at most 15 epochs; patience 4; seeds 0, 1, 2.
Shared training controls	Batch size 32; warmup 1 epoch(s); gradient clipping norm 1.
Optimizers	Baseline: Adam; searched recipes: AdamW.
Analytic benchmarks	10 dimensions; population 40 for population methods; 20,000 evaluations per run; 30 independent runs per method and function.

Table 6.2 lists the training settings. All choices are made on validation data; the test images are used only once, at the very end, after every recipe has been fixed. Each final recipe is trained three times with different random seeds to show how much results vary between runs. Speed (latency) is measured as the time to classify one image.

6.3 Benchmark test functions

Before using DE on the CNN, we checked it on five standard test functions [40]. Each has a known best value of zero. Sphere is a simple bowl; Rosenbrock is a long curved valley; Rastrigin, Ackley, and Griewank have many local traps. The tests use ten dimensions; the pictures show two.

$$f_{\text{Sphere}}(\mathbf{x}) = \sum_{i=1}^{D} x_i^2, \tag{6.1}$$

$$f_{\text{Rosenbrock}}(\mathbf{x}) = \sum_{i=1}^{D-1} \left[100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2\right], \tag{6.2}$$

$$f_{\text{Rastrigin}}(\mathbf{x}) = 10D + \sum_{i=1}^{D} \left[x_i^2 - 10\cos(2\pi x_i)\right], \tag{6.3}$$

$$f_{\text{Ackley}}(\mathbf{x}) = -20e^{-0.2\sqrt{D^{-1}\sum_i x_i^2}} - e^{D^{-1}\sum_i \cos(2\pi x_i)} + 20 + e, \tag{6.4}$$

$$f_{\text{Griewank}}(\mathbf{x}) = 1 + \frac{1}{4000}\sum_{i=1}^{D} x_i^2 - \prod_{i=1}^{D} \cos\left(\frac{x_i}{\sqrt{i}}\right). \tag{6.5}$$

Table 6.3: Search range and location of the best point for each test function.

Function	Domain per coordinate	Minimizer
Sphere	$[-5.12, 5.12]$	$x_i = 0$
Rosenbrock	$[-5, 10]$	$x_i = 1$
Rastrigin	$[-5.12, 5.12]$	$x_i = 0$
Ackley	$[-32.768, 32.768]$	$x_i = 0$
Griewank	$[-600, 600]$	$x_i = 0$

6.4 Fair optimiser comparison

Every method gets the same budget of 20,000 function evaluations and is run 30 times with different seeds, giving 750 runs and 15,000,000 evaluations in total.

The population methods start from 40 evenly spread points; random search samples points independently. DE uses $F = 0.6$ and $CR = 0.9$. jDE starts from random F and CR values and keeps the ones that work. GA uses tournament selection, blend crossover, small Gaussian mutations, and elitism. PSO uses the standard settings $w = 0.7298$ and $c_1 = c_2 = 1.49618$.

Curves show the median run with its middle-50% band; tables show mean, spread, median,

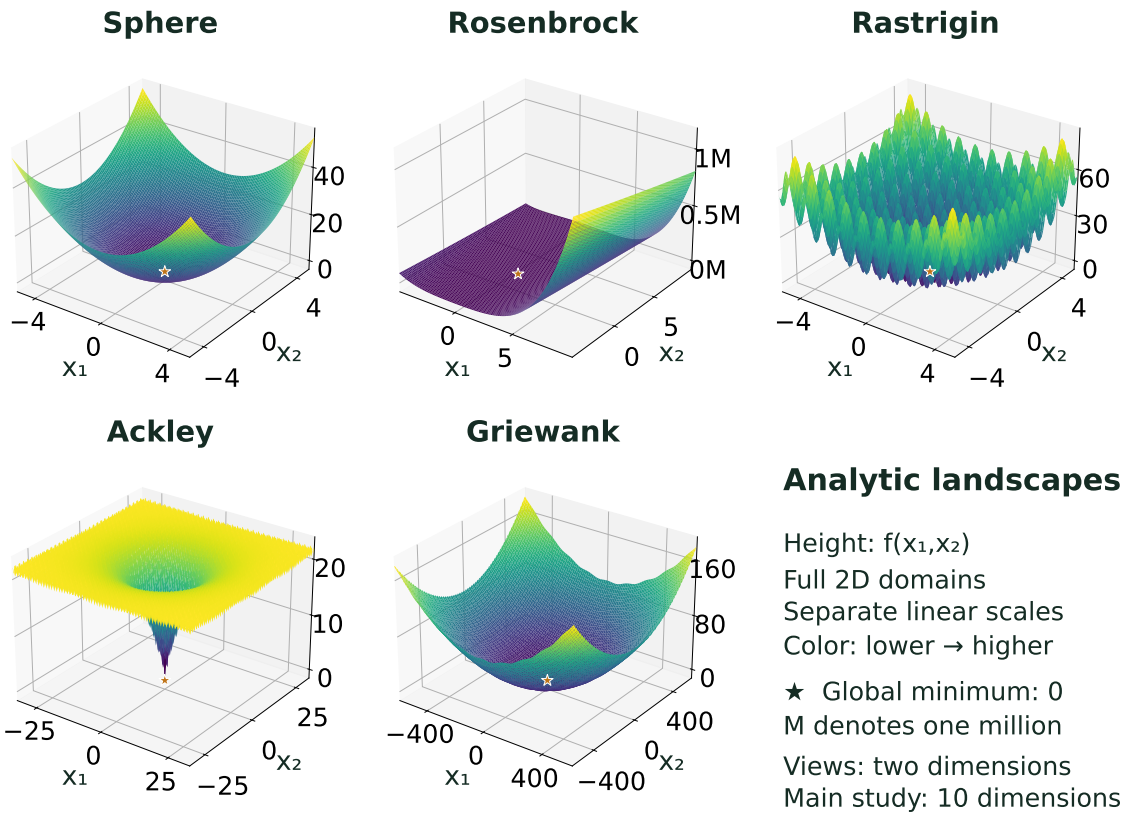


Figure 6.1: The five test functions drawn in two dimensions.

and best. A rank-sum test compares DE with each other method (Tables 8.1 and 8.2).

6.5 Corruption test

To see how the models cope with poorer photos, the seed-zero baseline and DE models were tested on the same test images with five kinds of damage: blur, noise, lower brightness and contrast, JPEG compression, and low light. Each was applied at three strengths, and the reported robustness score is the average accuracy over all of them.

CHAPTER 7

Results and Discussion

7.1 Checking DE on test functions

All five methods were given the same number of evaluations. On the ten-dimensional Sphere function, DE's median error is 2.59×10^{-15}, which is essentially zero. This confirms that our from-scratch DE works correctly.

Table 7.1: Final error on each test function: mean and standard deviation over all runs. Lower is better.

Function	DE	jDE	GA	PSO	Random Search
Sphere	4.786×10^{-15} $\pm$ 6.542×10^{-15}	3.113×10^{-23} $\pm$ 5.092×10^{-23}	0.001 121 $\pm$ 0.000 499 4	1.595×10^{-25} $\pm$ 3.598×10^{-25}	11.48 $\pm$ 2.71
Rosenbrock	0.7347 $\pm$ 0.8759	3.243 $\pm$ 0.91	9.206 $\pm$ 12.13	3.118 $\pm$ 2.094	8972 $\pm$ 3475
Rastrigin	28.91 $\pm$ 6.388	6.124×10^{-11} $\pm$ 2.315×10^{-10}	3.695 $\pm$ 1.916	7.429 $\pm$ 4.071	64.11 $\pm$ 7.415
Ackley	6.847×10^{-7} $\pm$ 5.662×10^{-7}	3.092×10^{-11} $\pm$ 2.28×10^{-11}	0.4568 $\pm$ 0.1393	3.517×10^{-12} $\pm$ 3.736×10^{-12}	16.46 $\pm$ 0.9384
Griewank	0.4422 $\pm$ 0.097 86	0.001 879 $\pm$ 0.004 593	0.6296 $\pm$ 0.1149	0.084 29 $\pm$ 0.033 51	40.42 $\pm$ 9.303

As expected, no single method wins on every function. DE converges reliably on Sphere and has the best median on the curved Rosenbrock valley. The self-adapting jDE does best on the trap-filled Rastrigin and Griewank functions. Errors close to 10^{-15} simply mean the method has reached the optimum.

7.2 How the DE search progressed

The DE search ran 72 trials. The population's average fitness rose quickly toward the best, and later generations made smaller refinements. Diversity shrank as the population gathered around good settings, but never reached zero, and children were still replacing parents in the last generation. The dashed baseline line in Figure 7.4 uses the same quick training setup, so the two are directly comparable.

7.3 Chosen recipes and training curves

The full-size retraining step picked the final DE recipe from its best few candidates. This recipe retrains the whole network, keeps the baseline's head size, and adds label smoothing and weight decay, with no extra RandAugment. Random search chose a recipe that retrains part of the network, uses a wider head, and applies stronger augmentation. Figure 7.8 agrees: trials that retrained more of the network and used less dropout tended to score higher.

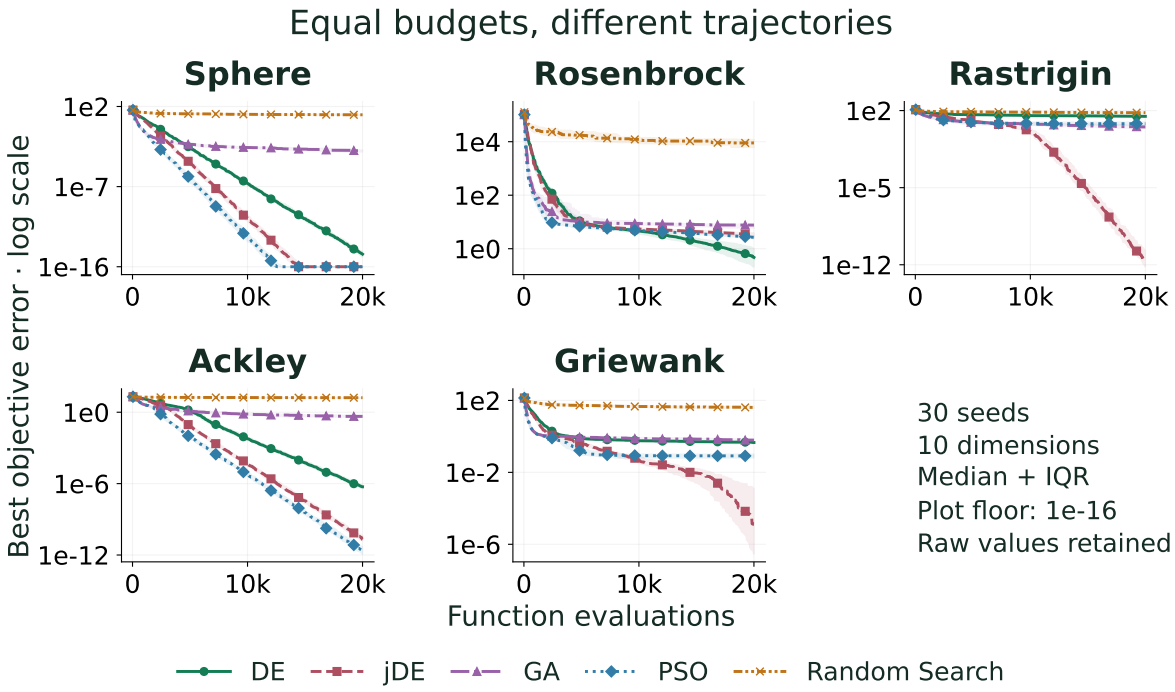


Figure 7.1: Best error found so far against the number of evaluations (median run with middle-50% band).

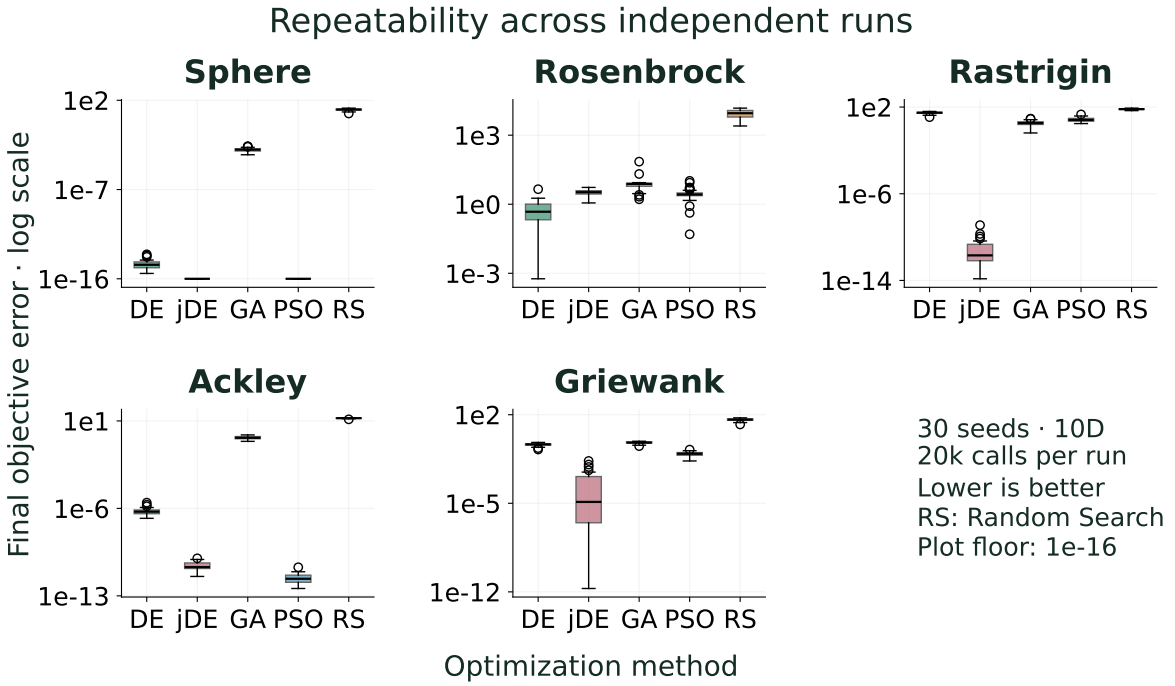


Figure 7.2: Spread of final errors over repeated runs for each method.

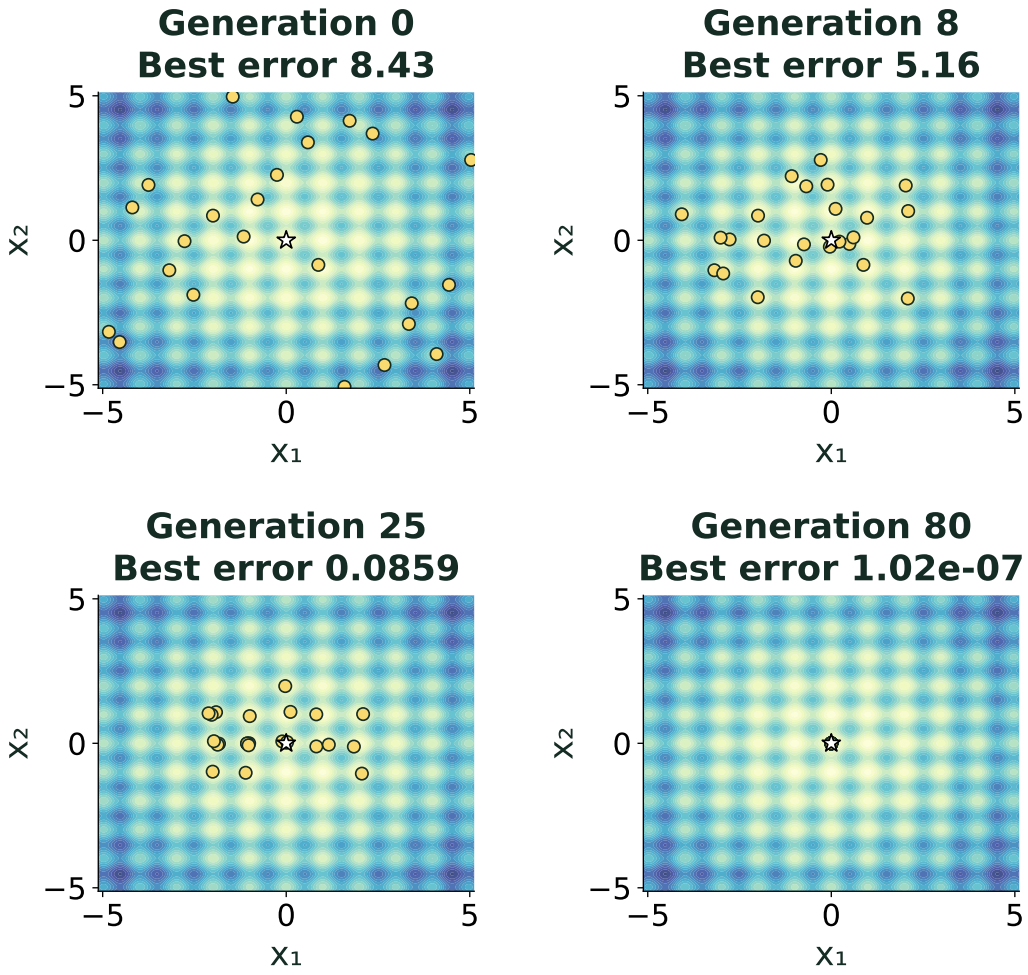


Figure 7.3: Snapshots of the DE population on the two-dimensional Rastrigin function over successive generations.

Both searched recipes reach a clearly higher validation macro-F1 than the baseline. Their loss values look higher only because label smoothing changes how the loss is measured; accuracy and macro-F1 are the fair comparison.

Table 7.2: Baseline and the final recipes chosen by DE and random search.

Parameter	Baseline	DE	Random search
Learning rate	0.001	0.001 257	0.000 656 3
Weight decay	0	0.000 416 8	2.568×10^{-6}
Dropout	0.2	0.2167341861	0.0293420375
Hidden units	256	256	512
Unfrozen blocks	0	9	6
Label smoothing	0.0	0.1056838563	0.0987632542
Augmentation magnitude	0	0	11

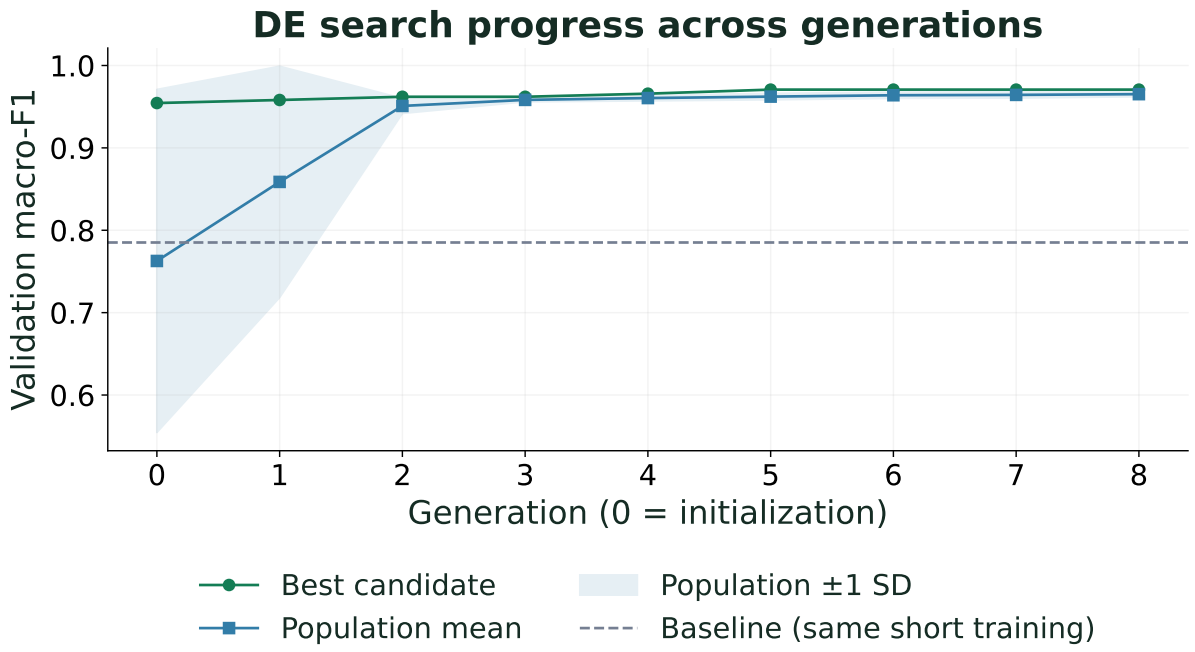


Figure 7.4: Best and average validation fitness per generation; the shaded band shows the spread of the population.

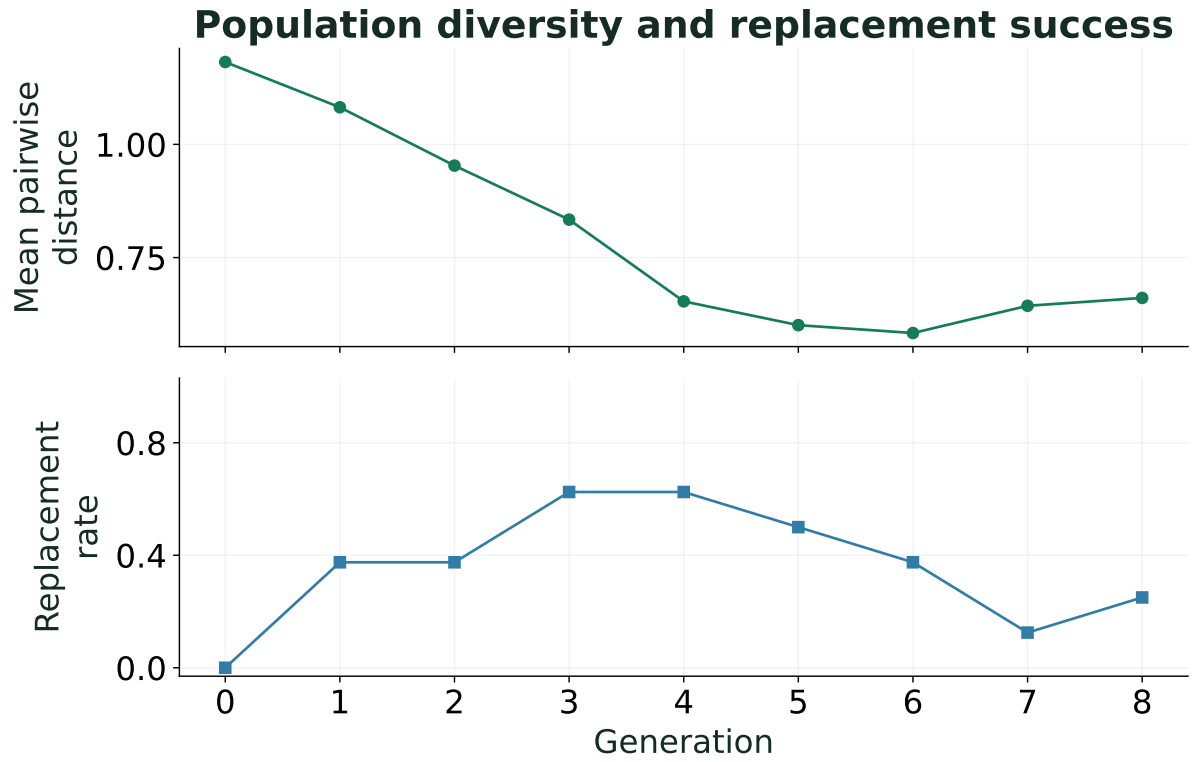


Figure 7.5: Population diversity and the share of children that replaced their parents in each generation.

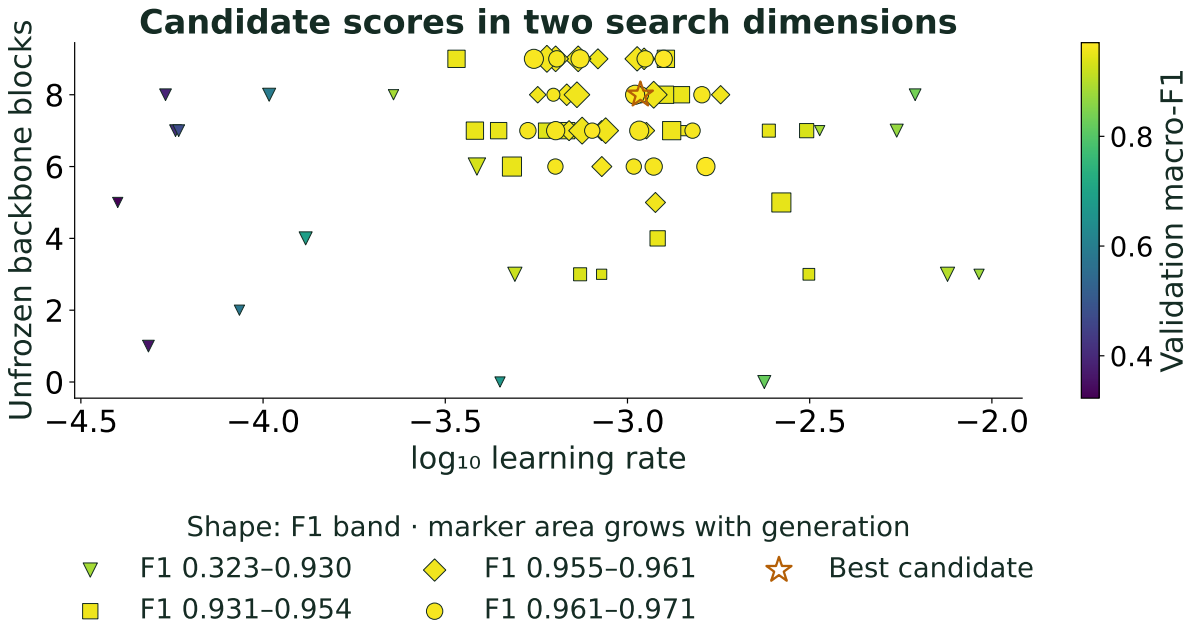


Figure 7.6: Every trial plotted by learning rate and fine-tuning depth, coloured by fitness.

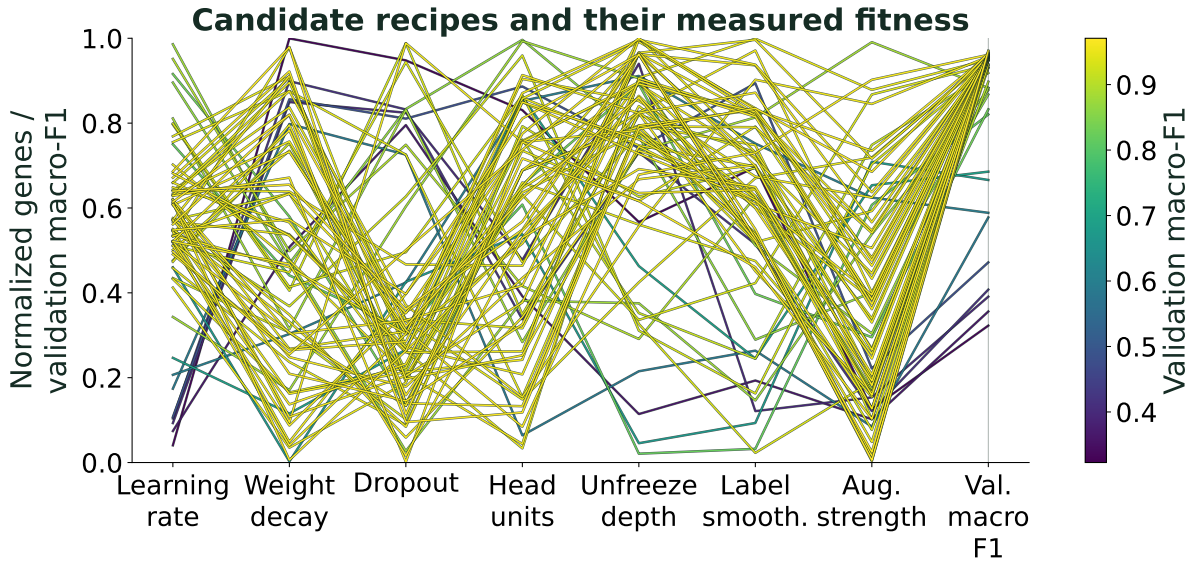


Figure 7.7: All trial recipes, one line each, across the seven genes.

7.4 Final test results

The DE recipe raises mean test accuracy from 97.43% to 99.85% and macro-F1 from 96.60% to 99.80%. Its results are also very stable across seeds, varying by only 0.06 percentage points. In other words, DE removes 94.29% of the baseline's mistakes. Random search reaches the same 99.85% accuracy and 99.80% macro-F1, which confirms that the improvement comes from tuning the training recipe.

Every class improves under DE. Early blight and Target Spot gain the most, although Early blight remains the hardest class. The baseline's most common mistakes were Target Spot confused with spider mites, Early blight with Late blight, and Target Spot with

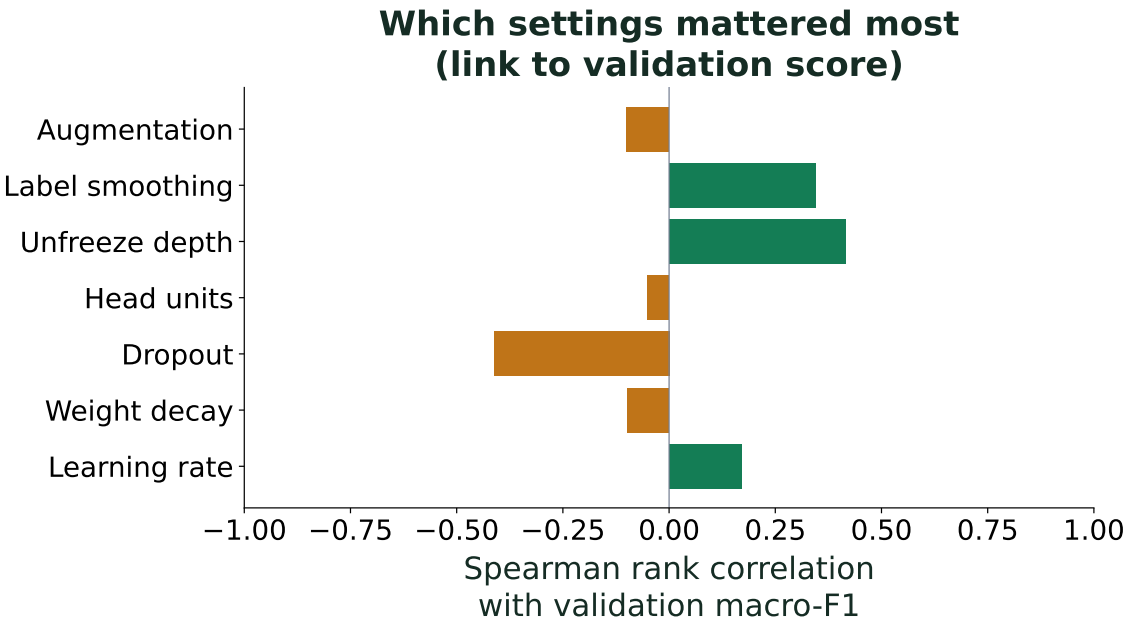


Figure 7.8: How strongly each gene was linked with higher fitness across all trials.

Table 7.3: Final test results: mean and standard deviation over three training seeds.

Model	Accuracy (%)	Precision (%)	Recall (%)	Macro-F1 (%)	ROC-AUC (%)
Baseline	97.43 ± 0.45	96.73 ± 0.61	96.53 ± 0.48	96.60 ± 0.54	99.94 ± 0.02
Random search	99.85 ± 0.04	99.80 ± 0.05	99.81 ± 0.07	99.80 ± 0.06	100.00 ± 0.00
DE-optimized	99.85 ± 0.06	99.82 ± 0.10	99.79 ± 0.10	99.80 ± 0.09	100.00 ± 0.00

healthy leaves. DE removes the Target Spot mistakes. Its few remaining errors are mostly between the two blight diseases, whose leaf spots can look alike.

7.5 Is the improvement real?

For the seed-zero models, the 95% confidence range for accuracy is [97.13, 98.24]% for the baseline and [99.63, 99.96]% for DE; for macro-F1 it is [96.15, 97.70]% and [99.48, 99.95]%. The ranges do not overlap. The McNemar test gives $p = 9.048 \times 10^{-15}$, which confirms that DE's improvement over the baseline is not due to chance.

Table 7.4: 95% bootstrap confidence intervals for the seed-zero models.

Model (seed 0)	Accuracy 95% CI (%)	Macro-F1 95% CI (%)
Baseline	[97.13, 98.24]	[96.15, 97.70]
DE-optimized	[99.63, 99.96]	[99.48, 99.95]
Random search	[99.74, 100.00]	[99.69, 100.00]

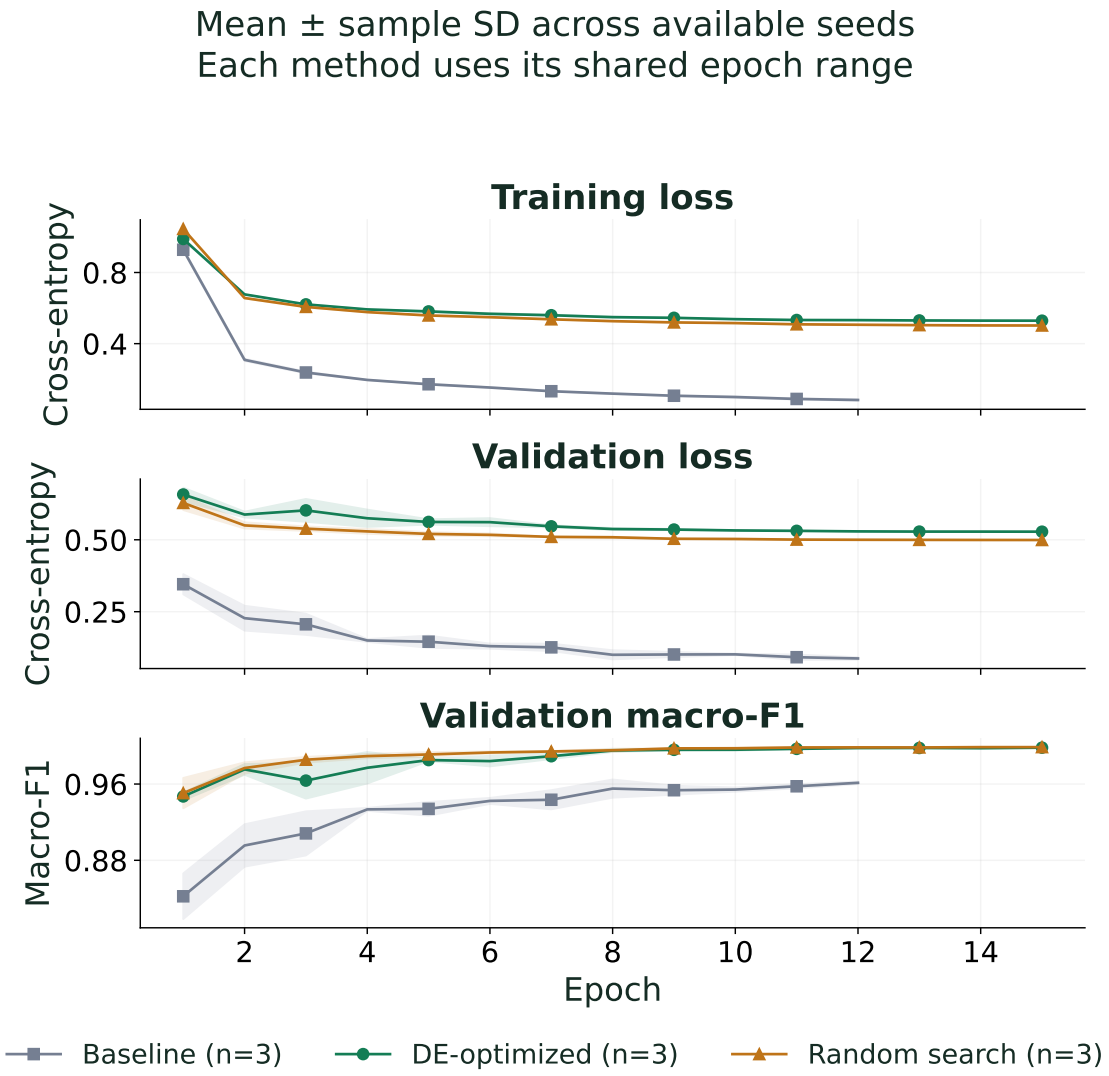


Figure 7.9: Training curves of the final models (mean and spread over seeds). Compare models by macro-F1 or accuracy, since label smoothing changes the loss scale.

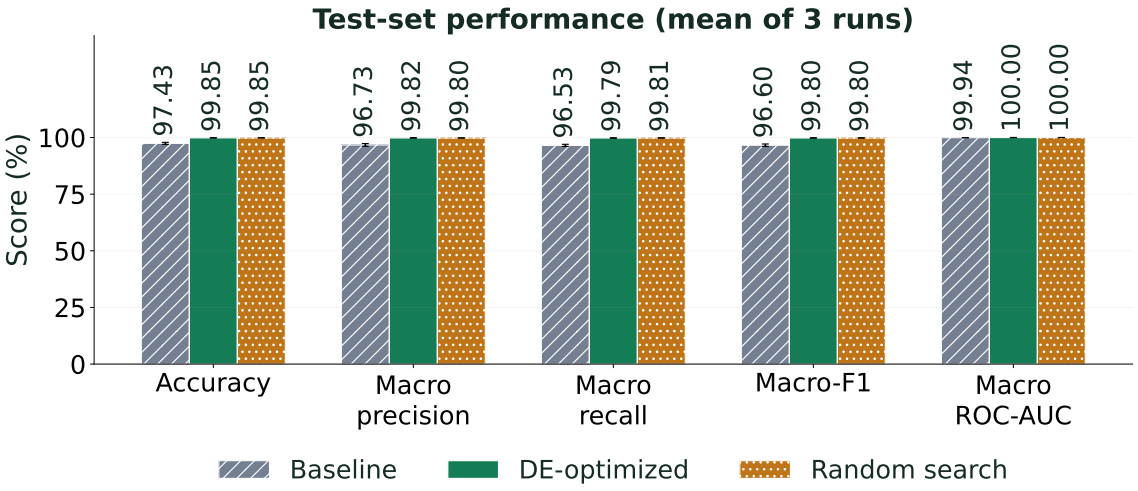


Figure 7.10: Test metrics of the baseline, DE, and random-search models, with variation across seeds. Higher is better.

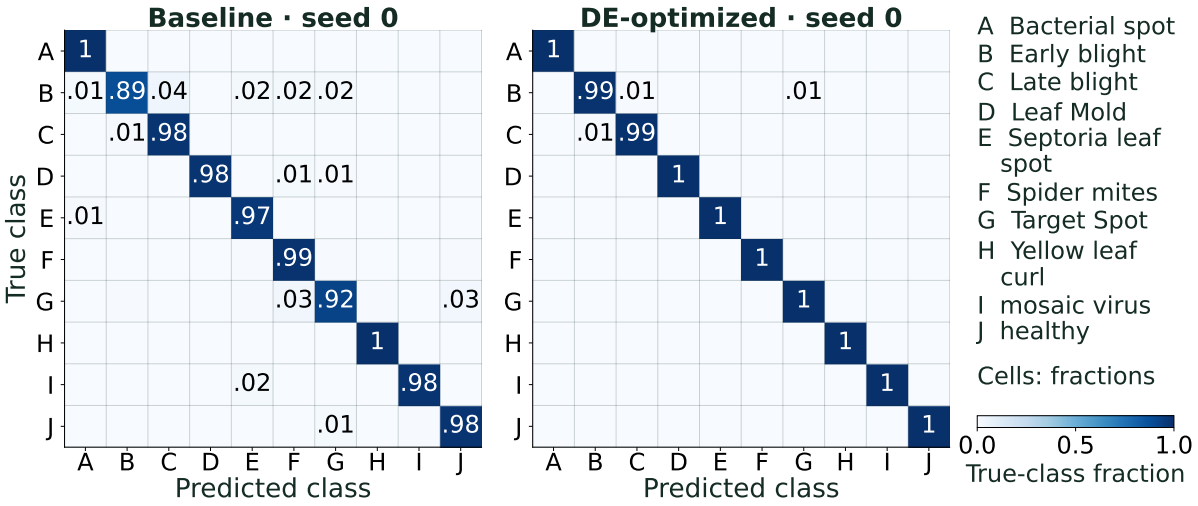


Figure 7.11: Confusion matrices for the seed-zero models. Each row is the true class; off-diagonal cells are mistakes.

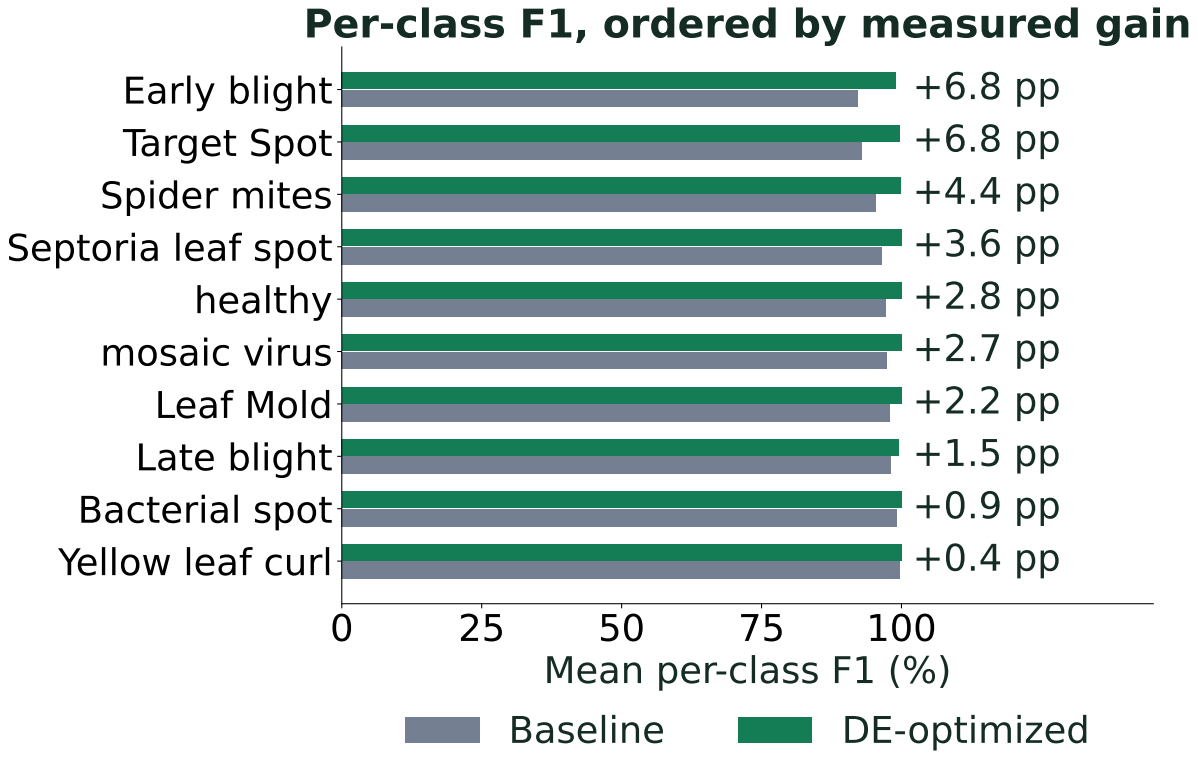


Figure 7.12: Average F1 for each class, sorted by how much DE improved over the baseline.

7.6 Robustness, confidence, and Grad-CAM

On the corruption test (Table 7.5 and figure 7.14), the DE model handles blur and JPEG compression better than the baseline, but is more affected by noise and strong lighting changes. Its average corruption accuracy is 69.19% against 85.77% for the baseline. Its calibration error is 9.23% against 0.81%; the reliability diagram shows that DE is mostly *under*-confident, meaning it is right more often than its confidence suggests, which fits with the softer targets from label smoothing. Since the search optimised only clean accuracy,

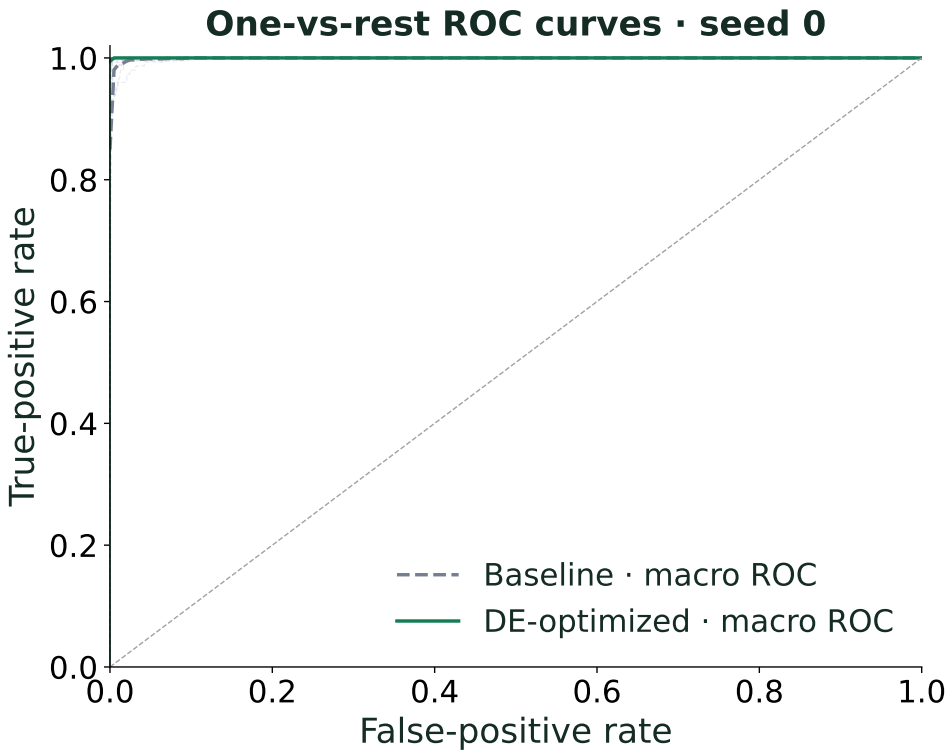


Figure 7.13: ROC curves for each class and their average, from the test predictions.

this shows that clean accuracy alone does not guarantee robustness. Adding robustness to the search goal is a natural next step (Chapter 11).

Table 7.5: Calibration error, average corruption accuracy, and speed of the seed-zero models.

Model	Mean corruption acc. (%)	ECE (%)	Latency (ms)
Baseline	85.77	0.81	4.71 ± 0.09
DE-optimized	69.19	9.23	4.74 ± 0.05

Both models classify all six example leaves correctly. The baseline's heat maps sometimes focus on leaf edges or the background, while DE's maps usually cover more of the leaf itself. Chapter 9 looks at three of these leaves in detail. The repository also contains a prediction gallery and a feature-space plot as extra material.

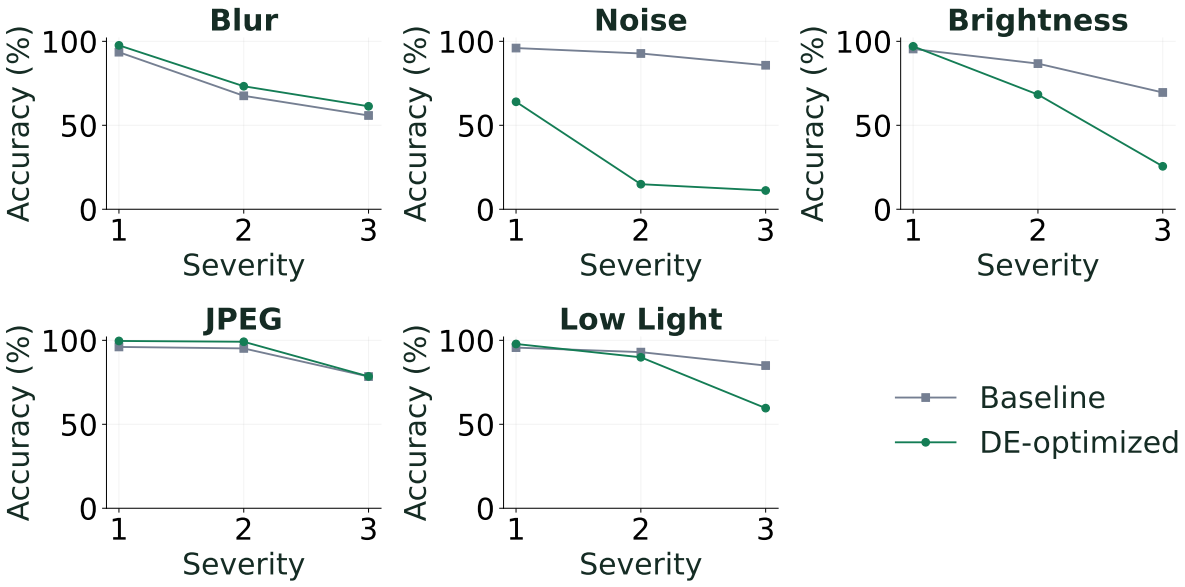


Figure 7.14: Accuracy under each type and strength of image damage.

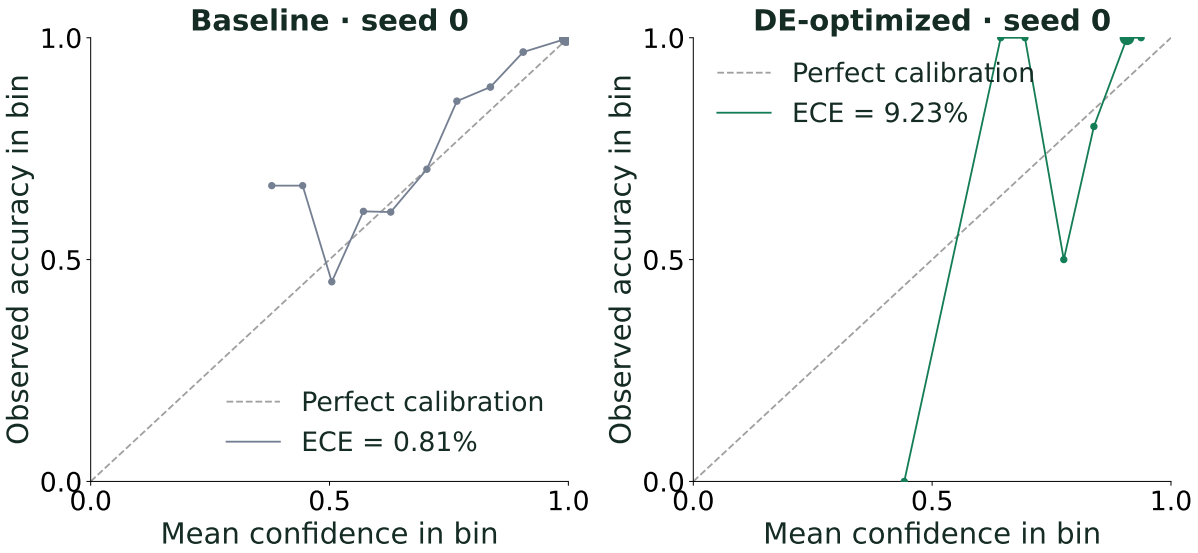


Figure 7.15: Reliability diagrams: how model confidence compares with actual correctness.

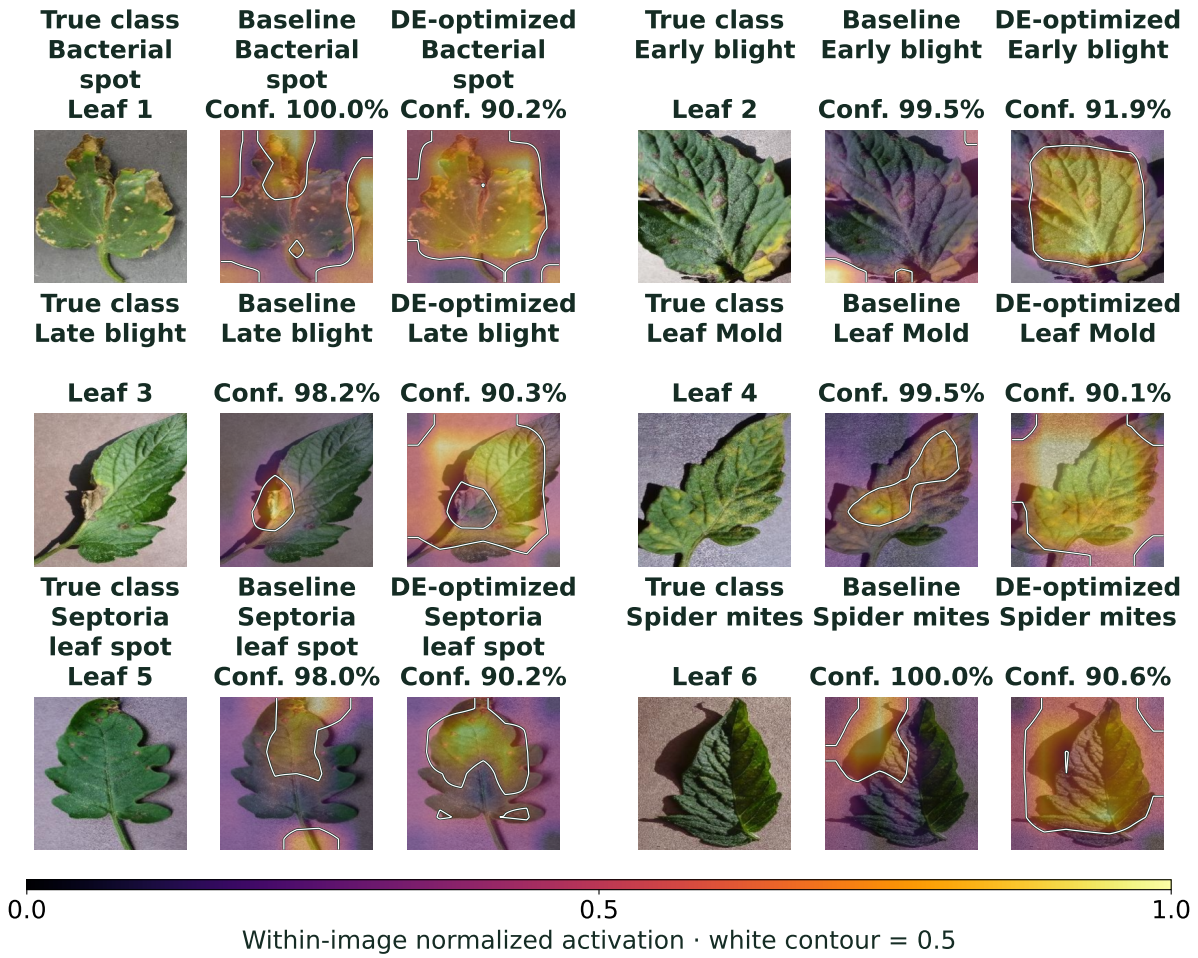


Figure 7.16: Grad-CAM heat maps for the first test image of six classes, for the seed-zero baseline and DE models. Brighter areas influenced the prediction more.

CHAPTER 8

Comparative Performance Analysis

8.1 DE versus random search

DE and random search used exactly the same settings space, data, fitness measure, number of trials, and final selection step. The only difference is how each one proposes new recipes, so this is a fair test of the search strategy itself.

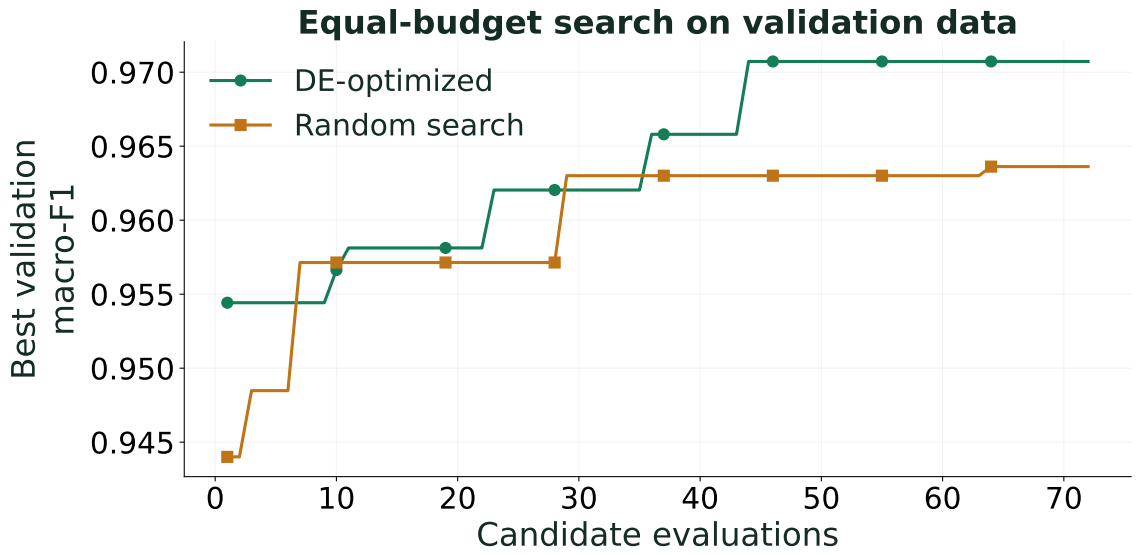


Figure 8.1: Best validation macro-F1 found so far against the number of trials, for DE and random search.

During the search, DE found higher-scoring recipes than random search (Figure 8.1), showing that its population steers trials toward good regions. After full-size retraining, random search's chosen candidate scored slightly higher on validation, and on the final test both strategies reach 99.85% accuracy and 99.80% macro-F1. So DE matched a strong random-search control on the final test while leading during the search itself. With only seven settings, 72 random trials already cover the space well; DE's guided search is likely to matter more in larger search spaces.

Against the baseline, the comparison covers the whole recipe, including the switch from Adam to AdamW. Published PlantVillage studies [4, 5, 6] and evolutionary-search studies [16, 19] use different classes, splits, and computing budgets, so they serve as context rather than a direct ranking.

8.2 Benchmark comparison and computing cost

On the test functions, the best method depends on the landscape: DE on Rosenbrock, PSO on Sphere and Ackley, and jDE on Rastrigin and Griewank. Random search has the

largest errors on every function, which shows that all the guided methods make good use of the problem's structure.

Table 8.1: Final benchmark errors: median and best over all runs.

Function	DE	jDE	GA	PSO	Random search
Sphere	2.59×10^{-15} 3.456×10^{-16}	1.27×10^{-23} 6.087×10^{-25}	0.001 055 0.000 321 7	2.59×10^{-26} 8.457×10^{-28}	11.66 4.67
Rosenbrock	0.4724 0.000 577 7	3.332 1.141	7.656 1.647	2.657 0.049 64	8991 2511
Rastrigin	30.01 11.68	2.053×10^{-12} 1.421×10^{-14}	3.298 0.4066	6.467 2.985	64.36 46.7
Ackley	5.464×10^{-7} 1.599×10^{-7}	2.02×10^{-11} 3.578×10^{-12}	0.447 0.2319	2.329×10^{-12} 3.877×10^{-13}	16.63 13.42
Griewank	0.4567 0.1779	1.286×10^{-5} 1.846×10^{-12}	0.6218 0.3308	0.082 41 0.022 15	41.02 17.04

Each cell: median (upper), best (lower); lower objective error is better.

Table 8.2: Rank-sum tests comparing DE with each other method; negative z favours DE.

Function	DE vs jDE	DE vs GA	DE vs PSO	DE vs random search
Sphere	$z = 6.653$ $p = 2.872 \times 10^{-11}$	$z = -6.653$ $p = 2.872 \times 10^{-11}$	$z = 6.653$ $p = 2.872 \times 10^{-11}$	$z = -6.653$ $p = 2.872 \times 10^{-11}$
Rosenbrock	$z = -6.121$ $p = 9.313 \times 10^{-10}$	$z = -6.55$ $p = 5.773 \times 10^{-11}$	$z = -5.485$ $p = 4.134 \times 10^{-8}$	$z = -6.653$ $p = 2.872 \times 10^{-11}$
Rastrigin	$z = 6.653$ $p = 2.872 \times 10^{-11}$	$z = 6.653$ $p = 2.872 \times 10^{-11}$	$z = 6.55$ $p = 5.773 \times 10^{-11}$	$z = -6.653$ $p = 2.872 \times 10^{-11}$
Ackley	$z = 6.653$ $p = 2.872 \times 10^{-11}$	$z = -6.653$ $p = 2.872 \times 10^{-11}$	$z = 6.653$ $p = 2.872 \times 10^{-11}$	$z = -6.653$ $p = 2.872 \times 10^{-11}$
Griewank	$z = 6.653$ $p = 2.872 \times 10^{-11}$	$z = -5.337$ $p = 9.44 \times 10^{-8}$	$z = 6.653$ $p = 2.872 \times 10^{-11}$	$z = -6.653$ $p = 2.872 \times 10^{-11}$

Two-sided Wilcoxon rank-sum tests over all recorded runs.

Negative z favors DE.

Table 8.3: Model size and speed; training time and prediction time are mean and standard deviation across seeds.

Model	Trainable params	Size (MiB)	Training (s)	Latency (ms)
Baseline	330,506	16.82	609.40 ± 83.61	4.71 ± 0.09
DE-optimized	4,338,054	16.82	1690.86 ± 83.86	4.74 ± 0.05
Random search	4,649,460	18.08	1321.46 ± 0.50	4.80 ± 0.07

Because the DE recipe retrains the whole network, it trains more parameters and takes longer to train. Its prediction time per image is almost the same as the baseline's. The complete benchmark study ran in 105.34 seconds.

CHAPTER 9

Applications and Case Studies

9.1 Offline diagnosis and farming use

The demo shows baseline and DE predictions side by side, their top-class scores, and Grad-CAM heat maps. Users can also add blur or noise to a leaf and watch the predictions change. A separate "Evolution Lab" animates how a DE population searches. These features suit classroom teaching and could support agricultural extension workers who advise farmers. Greenhouse monitoring, drone imaging, and disease-outbreak mapping are further possible uses; each would need new images and its own testing.

9.2 Three test leaves in detail

Figure 7.16 shows the first test image of each displayed class, so the examples were not hand-picked. Both models classify all of them correctly. Grad-CAM shows which regions each model relied on [23].

Early blight, Leaf 2. The leaf shows yellowing near the right edge. The baseline's heat map focuses on the lower edge and nearby background, while DE's strongest region covers much of the centre of the leaf. Both predict Early blight correctly; DE does so with lower confidence.

Late blight, Leaf 3. A discoloured patch runs along the left edge of the leaf. The baseline's attention sits close to that patch, while DE's map spreads more widely over the upper leaf and some background. Both predict Late blight correctly, using different image regions.

Spider mites, Leaf 6. The baseline's strongest attention includes background in the upper left and the leaf edge. DE highlights a larger part of the leaf. Both return the correct class, with different confidence levels.

The prediction gallery in the repository also shows DE's few remaining mistakes: Early blight taken for Late blight or Target Spot, and Late blight taken for Early blight. All of them involve the two blight diseases, matching the confusion matrix in Chapter 7.

9.3 PlantVillage Nuru: a real-world example

Mrisho et al. tested PlantVillage Nuru, a phone app that diagnoses cassava diseases offline [41]. They found that how images are taken, including how many leaves are examined, affects accuracy. This shows the practical points (phone, photo procedure, and user workflow) that matter when moving a tool like ours into the field.

9.4 DE beyond this project

DE is widely used in science and engineering, for example in design and energy problems [34, 35]. DEHB combines DE with early stopping of weak trials for tuning [19], and hybrid DE has been used to design CNNs [18].

CHAPTER 10

Conclusion

This project connected a from-scratch Differential Evolution implementation to a transfer-learning CNN for tomato leaf disease classification. Training settings were chosen on validation data only, and the final models were tested once on held-out images.

The cleaned dataset contains 18,146 images. The benchmark study completed 750 optimiser runs and 15,000,000 function evaluations. It confirmed that our DE works correctly and showed that each method has landscapes where it does best.

On the tomato task, DE raised mean test accuracy from 97.43% to 99.85% and macro-F1 from 96.60% to 99.80%, removing 94.29% of the baseline's errors. Every class improved, and the McNemar test ($p = 9.048 \times 10^{-15}$) confirms that the improvement is not due to chance. Random search with the same number of trials reached the same accuracy, which confirms that tuning the training recipe is what drives the gain; DE matched this strong control and led it during the search.

The objectives from Chapter 1 were met as follows:

1. The dataset was cleaned, documented, and split fairly, with the split lists saved in the repository.
2. DE was implemented from scratch, unit-tested, and validated on five test functions.
3. The baseline, DE, and random search were compared under the same rules and number of trials.
4. The final models were evaluated over three seeds, per class, with confidence intervals, a significance test, corruption tests, and Grad-CAM.
5. A working offline demo, this report, slides, and reproducible code were delivered.

Limitations. The results come from PlantVillage, whose photos were taken under controlled conditions, and from one search run per method. Testing on field photos and making the model more robust to poor-quality images are the main next steps, described in Chapter 11.

CHAPTER 11

Future Work

11.1 Test on field photos

The most important next step is to test the model on tomato leaf photos taken in real fields. PlantDoc and Field-PV show why this matters [8, 9]. Photos from different phones, places, disease stages, and leaves with more than one problem should be included. The model should also learn to say "I don't know" for leaves or plants it was not trained on.

11.2 Make the search faster

Multi-fidelity methods give more training time to promising recipes and stop weak ones early. DEHB is one example [19]. SHADE and L-SHADE adapt DE's own settings and population size while searching [37, 38]. Any such change should be compared with the same fair rules used here.

11.3 Multi-objective, robustness-aware search

Optimising clean accuracy alone does not guarantee robustness or good calibration. A natural extension is a multi-objective DE whose fitness also includes accuracy on corrupted validation images, calibration error, speed, and model size. Such a search returns a set of best trade-offs (a Pareto front) to choose from. Simple fixes such as temperature scaling for calibration and noise augmentation during training can also be added. For phone use, the model can be exported and timed on the target device; compact networks such as MobileNet offer further options [11, 12].

11.4 Stronger evidence and user feedback

Repeating the whole CNN search with different seeds would show how much the search itself varies. Plant experts could review the Grad-CAM maps. Feedback from farmers and extension workers could guide the collection of new, difficult images for future training and testing.

CHAPTER 12

References

[1] S. Savary, L. Willocquet, S. J. Pethybridge, P. Esker, N. McRoberts, and A. Nelson, "The global burden of pathogens and pests on major food crops," *Nature Ecology & Evolution*, vol. 3, no. 3, pp. 430–439, 2019. [Online]. Available: https://doi.org/10.1038/s41559-018-0793-y

[2] Ministry of Agriculture and Farmers Welfare, "Horticulture area and production estimates, released 17 march 2026," Press Information Bureau, Government of India, 2026, official government source; no DOI assigned. [Online]. Available: https://www.pib.gov.in/PressReleasePage.aspx?PRID=2241379&lang=1&reg=3

[3] D. P. Hughes and M. Salathe, "An open access repository of images on plant health to enable the development of mobile disease diagnostics," arXiv preprint, 2015. [Online]. Available: https://doi.org/10.48550/arxiv.1511.08060

[4] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using Deep Learning for Image-Based Plant Disease Detection," *Frontiers in Plant Science*, vol. 7, 2016. [Online]. Available: https://doi.org/10.3389/fpls.2016.01419

[5] K. P. Ferentinos, "Deep learning models for plant disease detection and diagnosis," *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, 2018. [Online]. Available: https://doi.org/10.1016/j.compag.2018.01.009

[6] E. C. Too, L. Yujian, S. Njuki, and L. Yingchun, "A comparative study of fine-tuning deep learning models for plant disease identification," *Computers and Electronics in Agriculture*, vol. 161, pp. 272–279, 2019. [Online]. Available: https://doi.org/10.1016/j.compag.2018.03.032

[7] A. Kamilaris and F. X. Prenafeta-Boldú, "Deep learning in agriculture: A survey," *Computers and Electronics in Agriculture*, vol. 147, pp. 70–90, 2018. [Online]. Available: https://doi.org/10.1016/j.compag.2018.02.016

[8] D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat, and N. Batra, "PlantDoc: A Dataset for Visual Plant Disease Detection," in *Proceedings of the 7th ACM IKDD CoDS and 25th COMAD*. ACM, 2020, pp. 249–253. [Online]. Available: https://doi.org/10.1145/3371158.3371196

[9] P. Gui, W. Dang, F. Zhu, and Q. Zhao, "Towards automatic field plant disease recognition," *Computers and Electronics in Agriculture*, vol. 191, p. 106523, 2021. [Online]. Available: https://doi.org/10.1016/j.compag.2021.106523

[10] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," arXiv preprint, 2019. [Online]. Available: https://doi.org/10.48550/arxiv.1905.11946

[11] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," arXiv preprint, 2018. [Online]. Available: https://doi.org/10.48550/arxiv.1801.04381

[12] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, "Searching for MobileNetV3," arXiv preprint, 2019. [Online]. Available: https://doi.org/10.48550/arxiv.1905.02244

[13] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" arXiv preprint, 2014. [Online]. Available: https://doi.org/10.48550/arxiv.1411.1792

[14] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian Optimization of Machine Learning Algorithms," arXiv preprint, 2012. [Online]. Available: https://doi.org/10.48550/arxiv.1206.2944

[15] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, "Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization," arXiv preprint, 2016. [Online]. Available: https://doi.org/10.48550/arxiv.1603.06560

[16] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, "Regularized Evolution for Image Classifier Architecture Search," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, pp. 4780–4789, 2019. [Online]. Available: https://doi.org/10.1609/aaai.v33i01.33014780

[17] B. Wang, Y. Sun, B. Xue, and M. Zhang, "Evolving Deep Convolutional Neural Networks by Variable-Length Particle Swarm Optimization for Image Classification," in *2018 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2018, pp. 1–8. [Online]. Available: https://doi.org/10.1109/cec.2018.8477735

[18] ——, "A Hybrid Differential Evolution Approach to Designing Deep Convolutional Neural Networks for Image Classification," in *AI 2018: Advances in Artificial Intelligence*. Springer International Publishing, 2018, pp. 237–250. [Online]. Available: https://doi.org/10.1007/978-3-030-03991-2_24

[19] N. Awad, N. Mallik, and F. Hutter, "DEHB: Evolutionary Hyberband for Scalable, Robust and Efficient Hyperparameter Optimization," in *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence Organization, 2021, pp. 2147–2153. [Online]. Available: https://doi.org/10.24963/ijcai.2021/296

[20] B. Efron, "Bootstrap Methods: Another Look at the Jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979. [Online]. Available: https://doi.org/10.1214/aos/1176344552

[21] T. G. Dietterich, "Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998. [Online]. Available: https://doi.org/10.1162/089976698300017197

[22] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," arXiv preprint, 2017. [Online]. Available: https://doi.org/10.48550/arxiv.1706.04599

[23] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization," in *2017 IEEE International Conference on Computer Vision (ICCV)*. IEEE, 2017, pp. 618–626. [Online]. Available: https://doi.org/10.1109/iccv.2017.74

[24] D. Hendrycks and T. Dietterich, "Benchmarking Neural Network Robustness to Common Corruptions and Perturbations," arXiv preprint, 2019. [Online]. Available: https://doi.org/10.48550/arxiv.1903.12261

[25] J. H. Holland, *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. The MIT Press, 1992. [Online]. Available: https://doi.org/10.7551/mitpress/1090.001.0001

[26] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of ICNN'95 - International Conference on Neural Networks*, vol. 4. IEEE, 1995, pp. 1942–1948. [Online]. Available: https://doi.org/10.1109/icnn.1995.488968

[27] M. Dorigo, V. Maniezzo, and A. Colorni, "Ant system: optimization by a colony of cooperating agents," *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, vol. 26, no. 1, pp. 29–41, 1996. [Online]. Available: https://doi.org/10.1109/3477.484436

[28] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by Simulated Annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983. [Online]. Available: https://doi.org/10.1126/science.220.4598.671

[29] X.-S. Yang, "Firefly Algorithms for Multimodal Optimization," in *Stochastic Algorithms: Foundations and Applications*. Springer Berlin Heidelberg, 2009, pp. 169–178. [Online]. Available: https://doi.org/10.1007/978-3-642-04944-6_14

[30] D. Simon, "Biogeography-Based Optimization," *IEEE Transactions on Evolutionary Computation*, vol. 12, no. 6, pp. 702–713, 2008. [Online]. Available: https://doi.org/10.1109/tevc.2008.919004

[31] D. Karaboga and B. Basturk, "A powerful and efficient algorithm for numerical function optimization: artificial bee colony (ABC) algorithm," *Journal of Global Optimization*, vol. 39, no. 3, pp. 459–471, 2007. [Online]. Available: https://doi.org/10.1007/s10898-007-9149-x

[32] S. Mirjalili, S. M. Mirjalili, and A. Lewis, "Grey Wolf Optimizer," *Advances in Engineering Software*, vol. 69, pp. 46–61, 2014. [Online]. Available: https://doi.org/10.1016/j.advengsoft.2013.12.007

[33] R. Storn and K. Price, "Differential Evolution – A Simple and Efficient Heuristic for global Optimization over Continuous Spaces," *Journal of Global Optimization*, vol. 11, no. 4, pp. 341–359, 1997. [Online]. Available: https://doi.org/10.1023/a:1008202821328

[34] S. Das and P. N. Suganthan, "Differential Evolution: A Survey of the State-of-the-Art," *IEEE Transactions on Evolutionary Computation*, vol. 15, no. 1, pp. 4–31, 2011. [Online]. Available: https://doi.org/10.1109/tevc.2010.2059031

[35] S. Das, S. S. Mullick, and P. Suganthan, "Recent advances in differential evolution – An updated survey," *Swarm and Evolutionary Computation*, vol. 27, pp. 1–30, 2016. [Online]. Available: https://doi.org/10.1016/j.swevo.2016.01.004

[36] J. Brest, S. Greiner, B. Boskovic, M. Mernik, and V. Zumer, "Self-Adapting Control Parameters in Differential Evolution: A Comparative Study on Numerical Benchmark Problems," *IEEE Transactions on Evolutionary Computation*, vol. 10, no. 6, pp. 646–657, 2006. [Online]. Available: https://doi.org/10.1109/tevc.2006.872133

[37] R. Tanabe and A. Fukunaga, "Success-history based parameter adaptation for Differential Evolution," in *2013 IEEE Congress on Evolutionary Computation*. IEEE, 2013, pp. 71–78. [Online]. Available: https://doi.org/10.1109/cec.2013.6557555

[38] R. Tanabe and A. S. Fukunaga, "Improving the search performance of SHADE using linear population size reduction," in *2014 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2014, pp. 1658–1665. [Online]. Available: https://doi.org/10.1109/cec.2014.6900380

[39] K. Sörensen, "Metaheuristics—the metaphor exposed," *International Transactions in Operational Research*, vol. 22, no. 1, pp. 3–18, 2015. [Online]. Available: https://doi.org/10.1111/itor.12001

[40] M. Jamil and X. S. Yang, "A literature survey of benchmark functions for global optimisation problems," *International Journal of Mathematical Modelling and Numerical Optimisation*, vol. 4, no. 2, p. 150, 2013. [Online]. Available: https://doi.org/10.1504/ijmmno.2013.055204

[41] L. M. Mrisho, N. A. Mbilinyi, M. Ndalahwa, A. M. Ramcharan, A. K. Kehs, P. C. McCloskey, H. Murithi, D. P. Hughes, and J. P. Legg, "Accuracy of a Smartphone-Based Object Detection Model, PlantVillage Nuru, in Identifying the Foliar Symptoms of the Viral Diseases of Cassava–CMD and CBSD," *Frontiers in Plant Science*, vol. 11, 2020. [Online]. Available: https://doi.org/10.3389/fpls.2020.590889

CHAPTER 13

GitHub Repo Link

https://github.com/Suraj-B12/BIO-INSPIRED-ALGO---AAT

The report tables were generated from source revision `5e28dddf8220`. The repository contains the configuration, numbered scripts, tests, saved results, figure generators, report sources, slide generator, and offline demo. The raw dataset and temporary training files are not stored in the repository.

Table 13.1: Repository map.

Location	Purpose
`bioalgo/`	Data and model helpers, training, DE and benchmark algorithms
`scripts/`	Numbered data, experiment, evaluation, and output scripts
`tests/`	Tests for the DE operators, data, model, demo, and a quick end-to-end run
`results/`	Saved experiment results
`figures/`	Plots generated from the saved results
`report/`, `slides/`	Report and presentation with their sources
`demo/`	Offline diagnosis demo and Evolution Lab

After creating a Python environment and installing the listed packages, the whole pipeline runs with `python run_all.py`. A quick test run uses `python run_all.py -smoke`. Figures can be regenerated from the saved results without retraining using `python scripts/09_figures.py`, and tables with `python scripts/10_make_tables.py`. Start the demo with `python demo/serve.py` and open the local address it prints.

The code runs on macOS, Linux, and Windows, and the repository includes a continuous-integration workflow for all three. Running the full pipeline downloads the dataset and pretrained weights once; after that, the demo works fully offline.